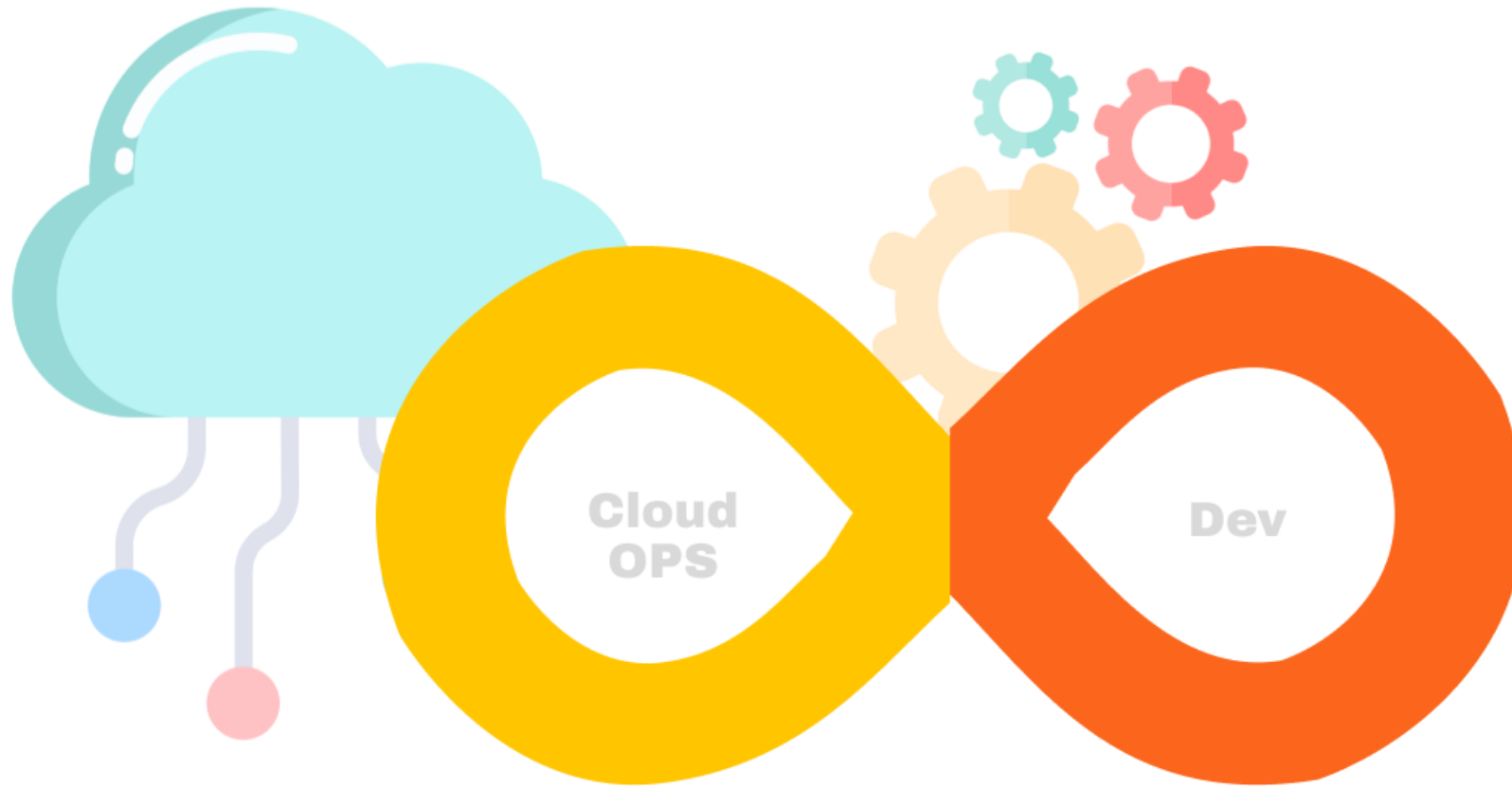




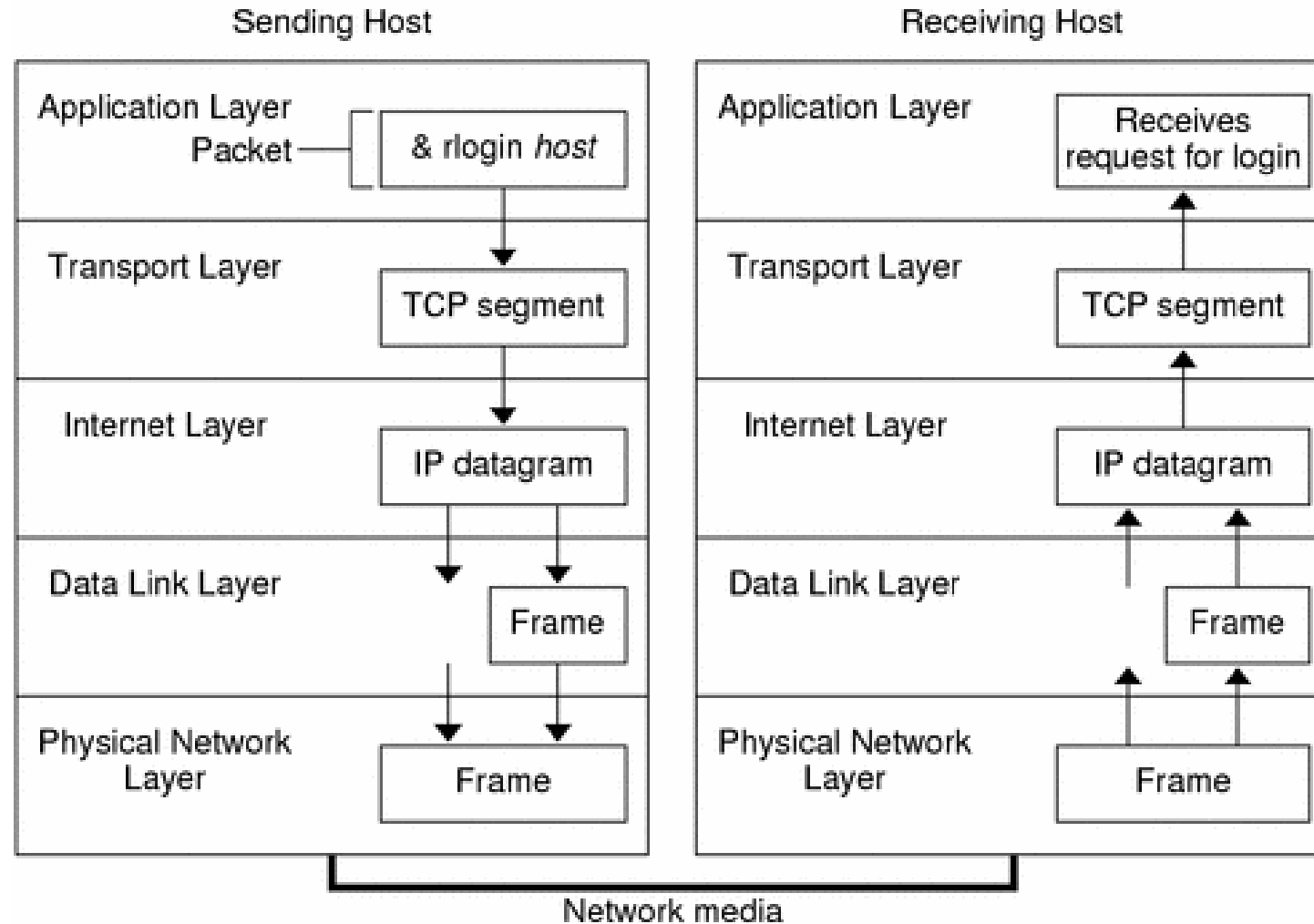
Containerization

Muhammad Ali

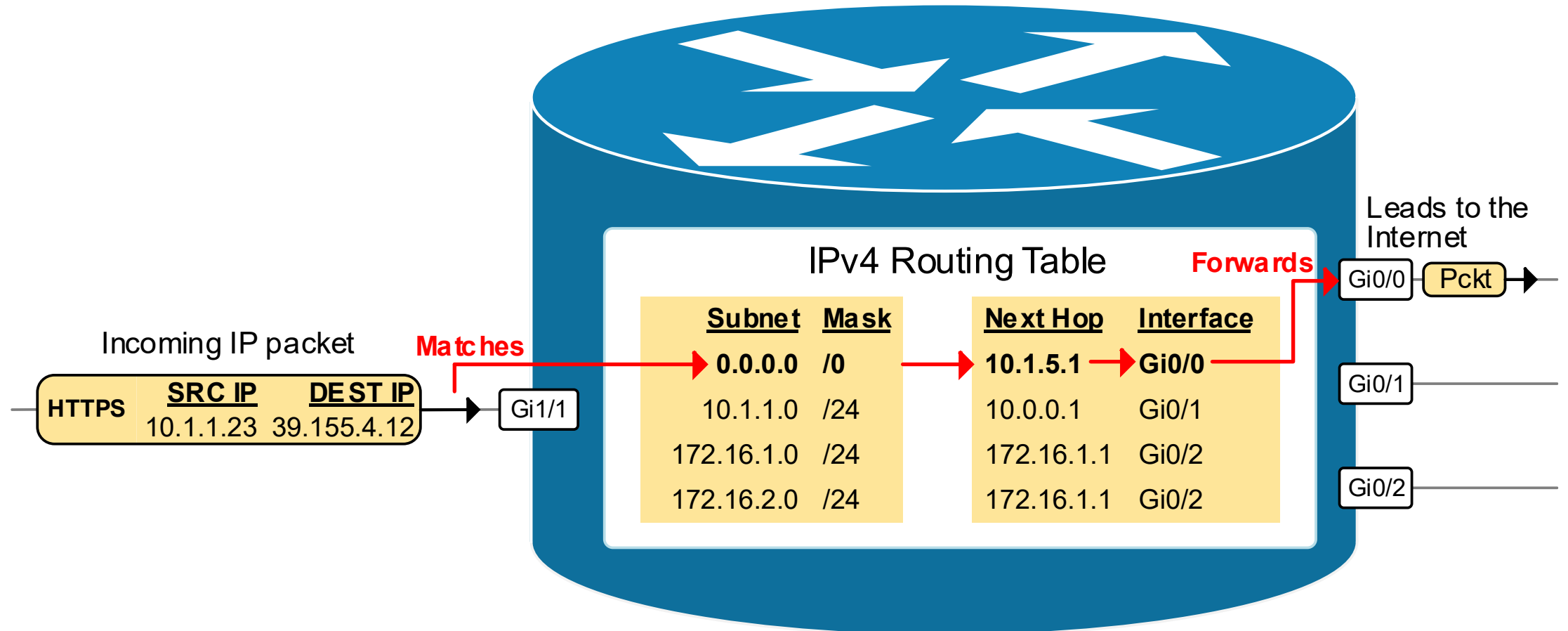
Course : DevOps and Cloud Computing Foundations
Department of Computer Science, SCIT, Beaconhouse National University
(BNU)

Networking Basics

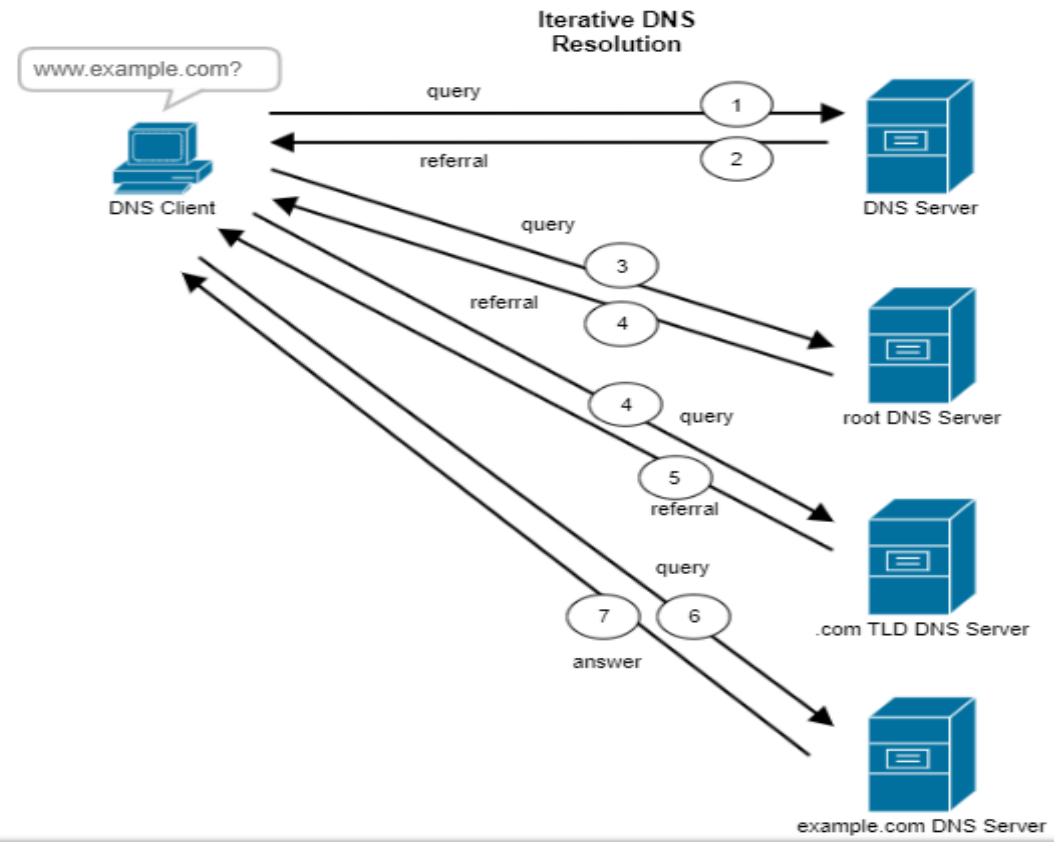
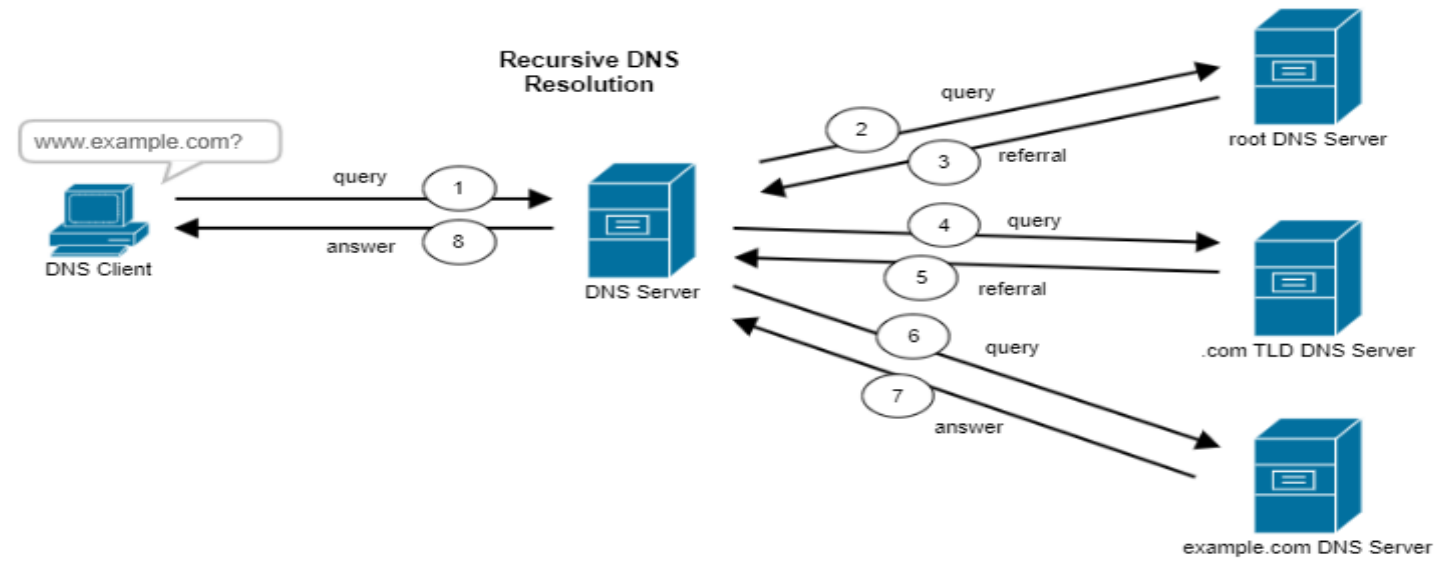
How TCP/IP handles data communication



Routing - Default routing, routing tables



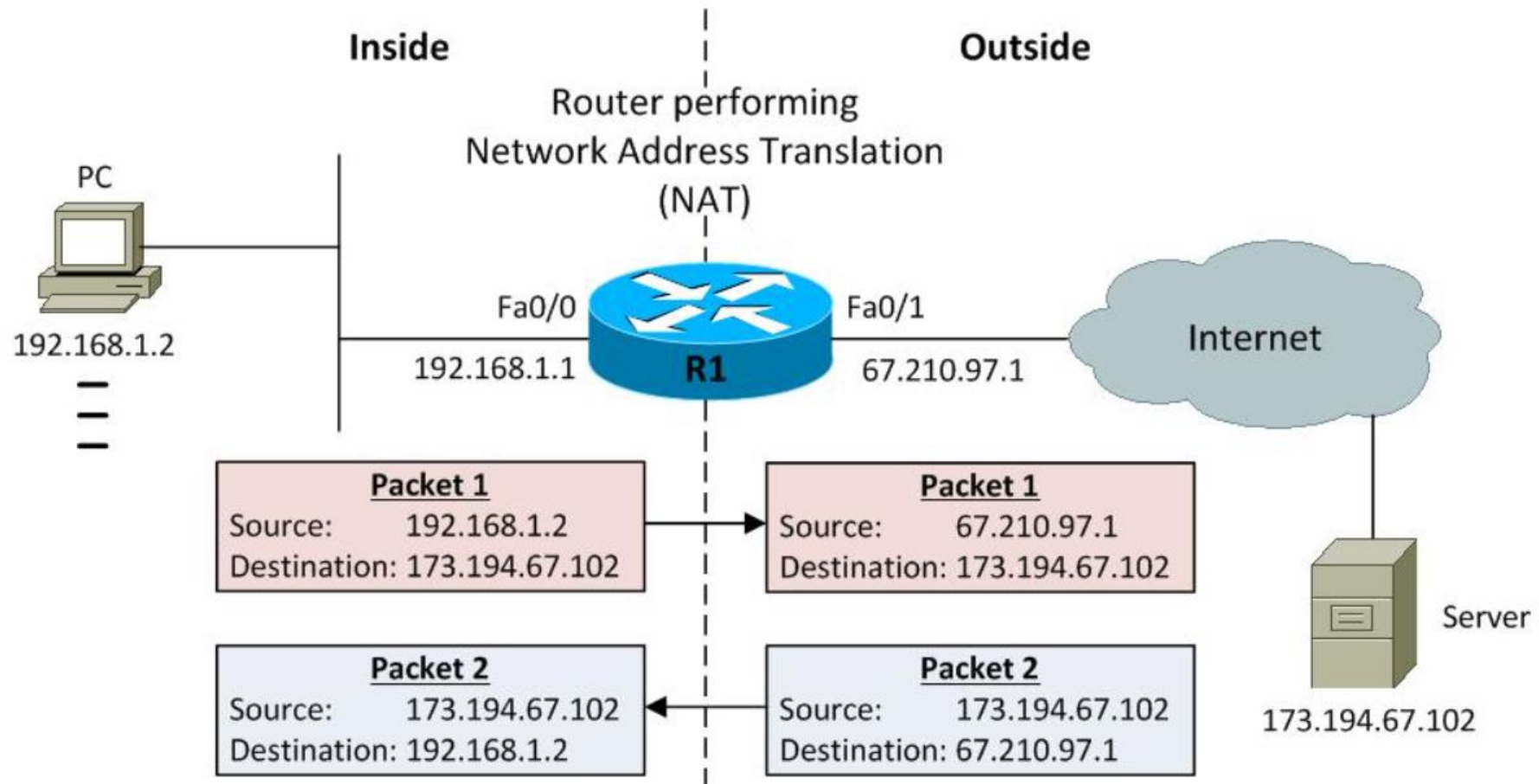
DNS - Fundamental



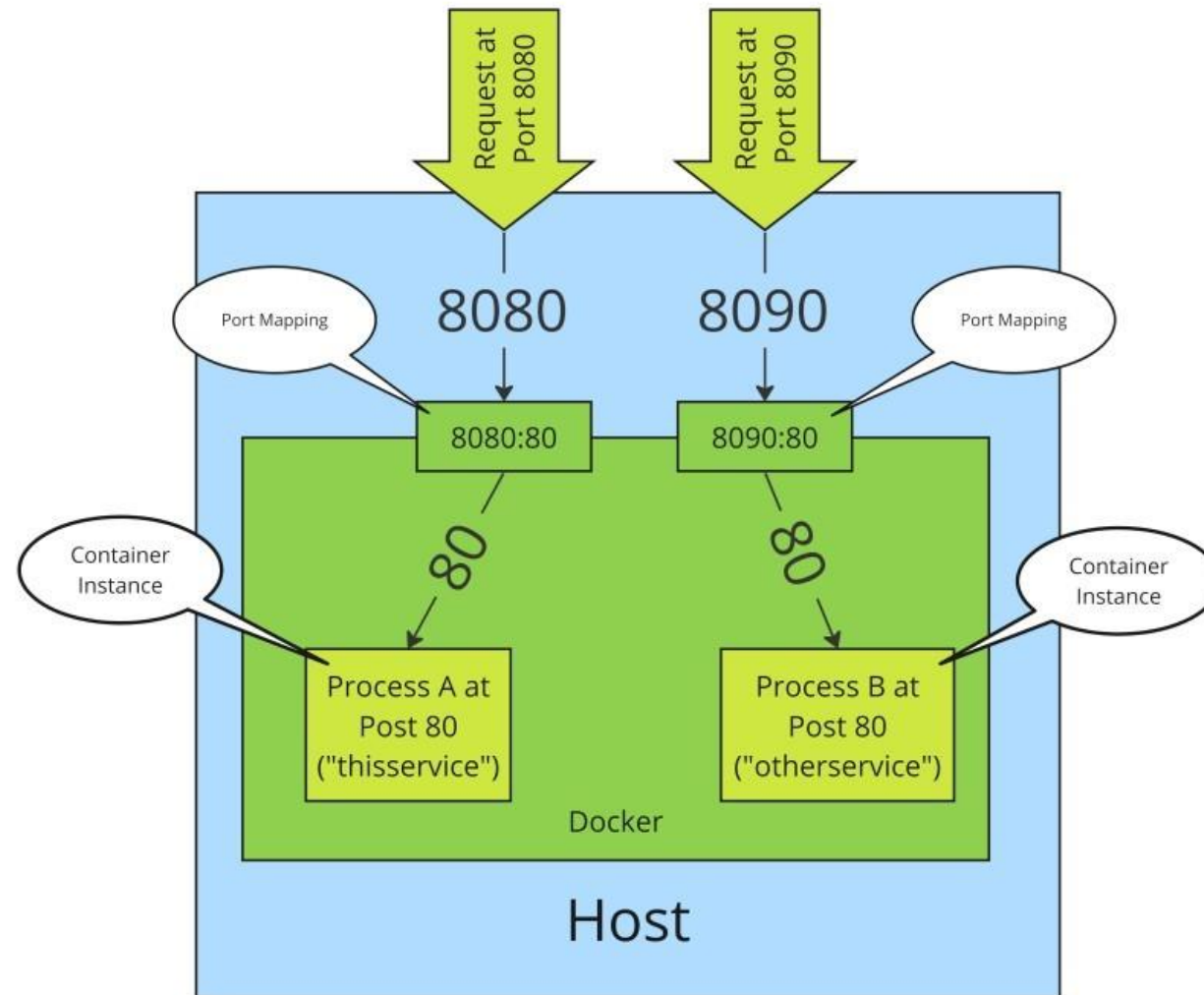
localhost vs LAN binding (127.0.0.1 vs 0.0.0.0)?

- *Bind to 127.0.0.1 → local-only (safest default for dev)*
- *Bind to 0.0.0.0 → network-accessible (needed for LAN / containers / reverse proxies)*

NAT – Network Address Translation



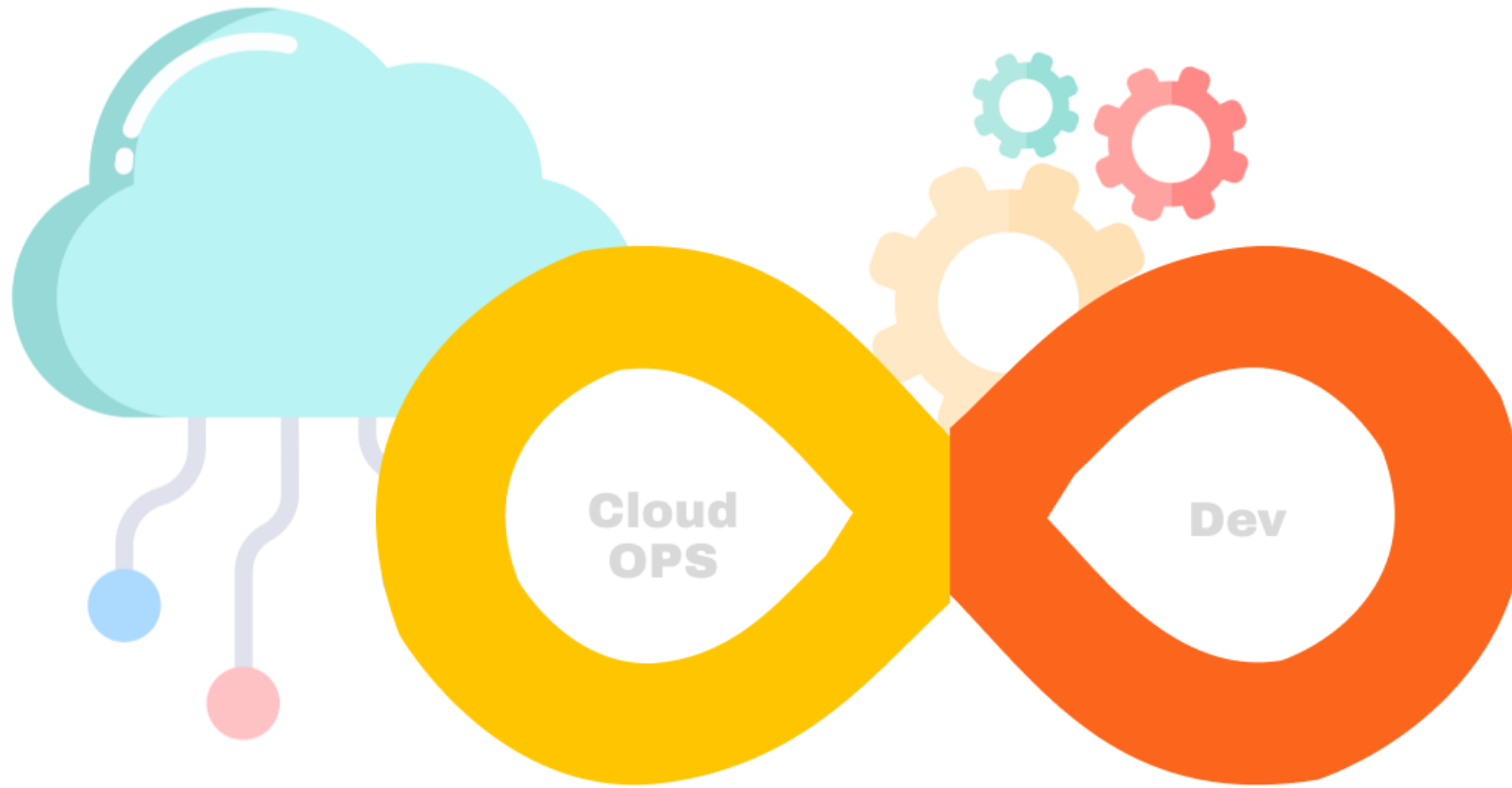
Port Binding – Port Forwarding



OK THATS COOL



I UNDERSTAND



Containerization

Muhammad Ali

Course : DevOps and Cloud Computing Foundations
Department of Computer Science, SCIT, Beaconhouse National University
(BNU)



Course Module: Containerization & Docker

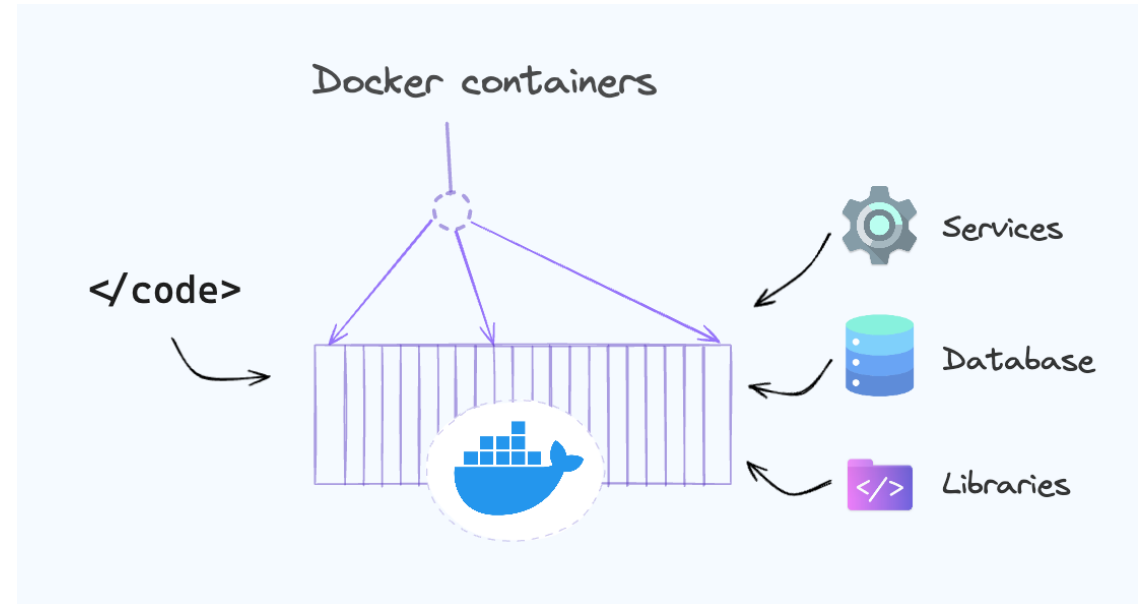
Module Structure

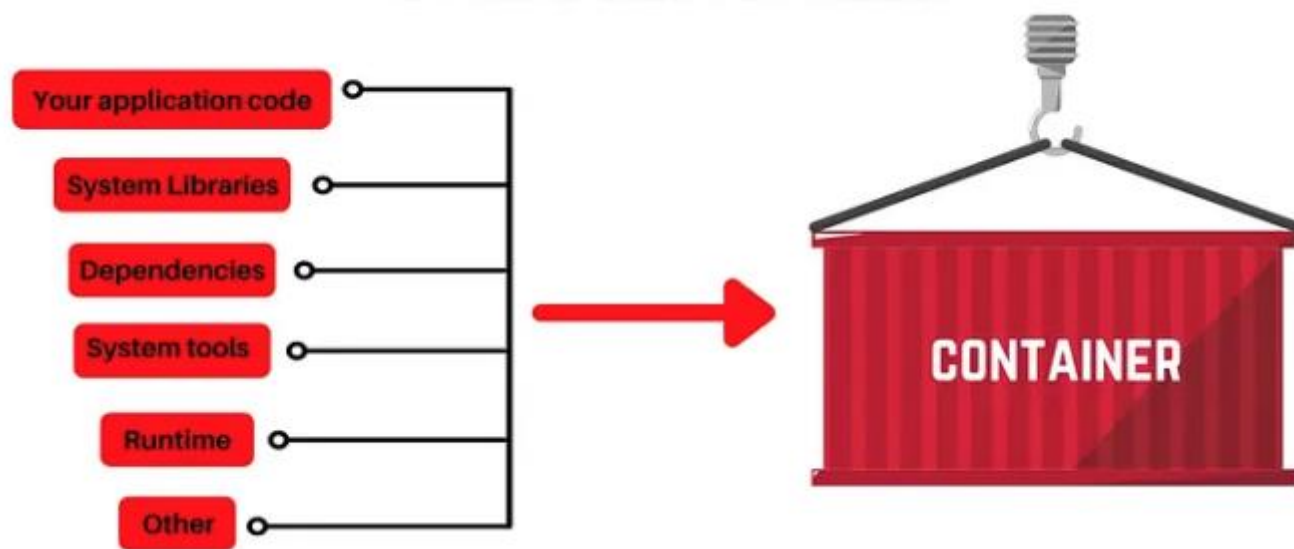
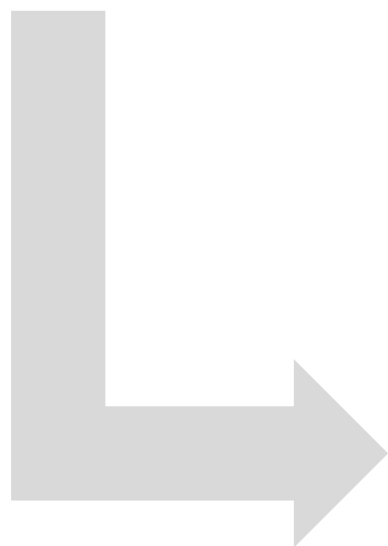
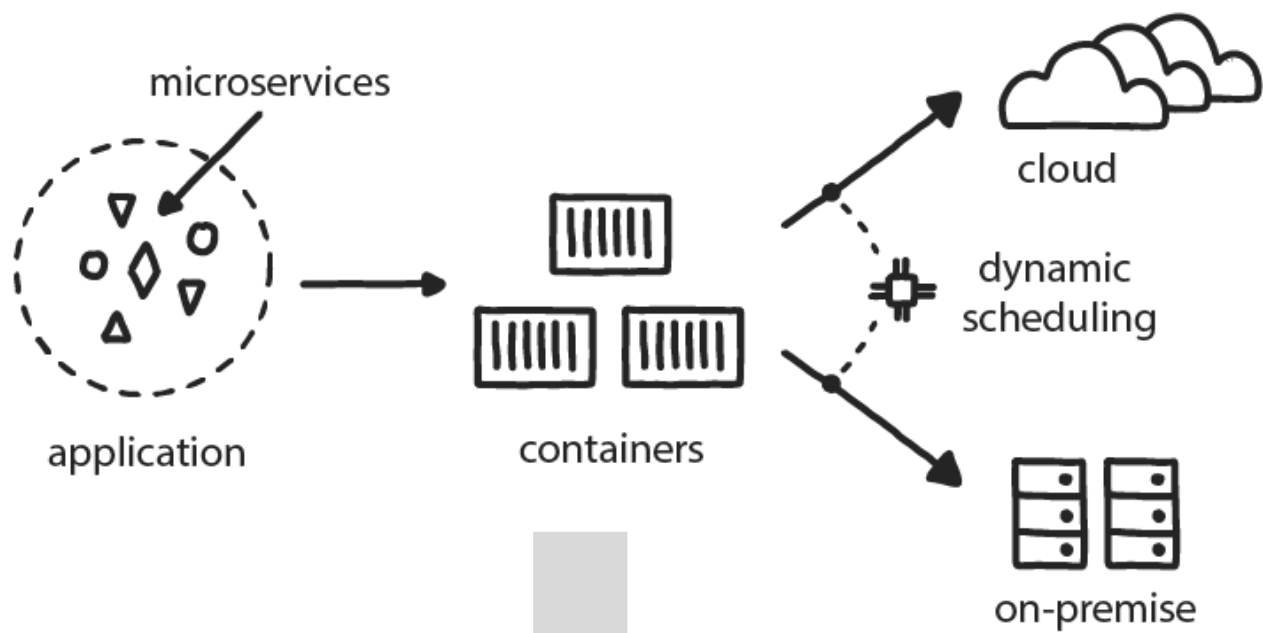
- **Part 1 — Understanding Containerization**
Concepts, evolution, importance, and alternatives.
- **Part 2 — Deep Dive into Docker**
Architecture, usage, commands, workflows, and best practices.



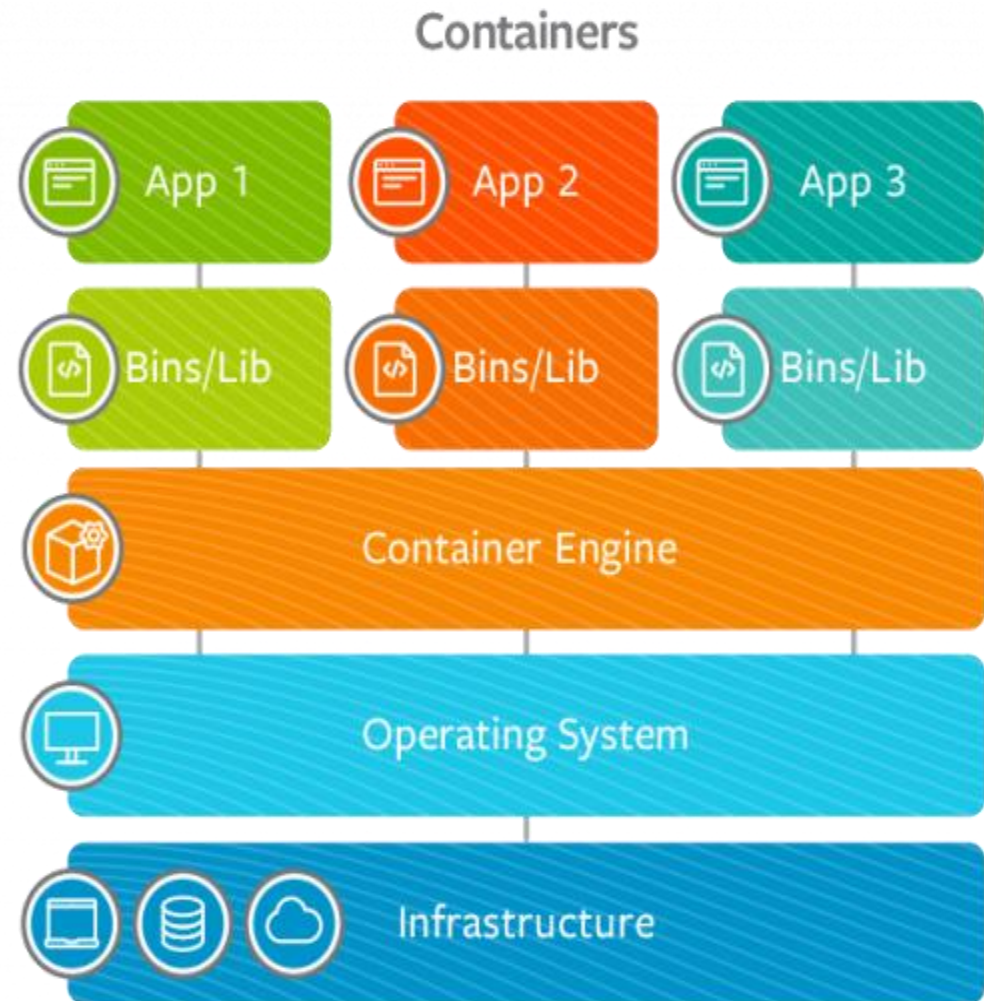
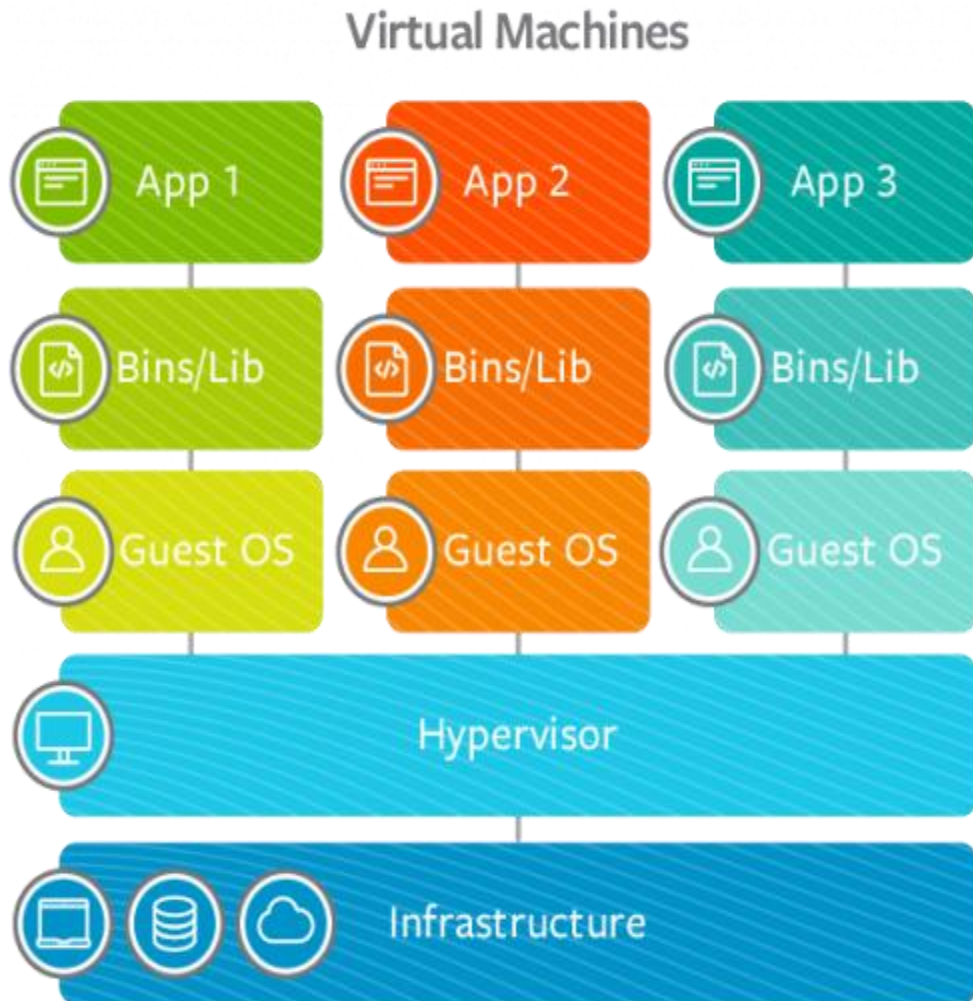
What is Containerization

Containerization is a lightweight form of virtualization that enables applications and their dependencies to be packaged into containers, making them portable and consistent across environments





Virtualization Vs Containerization



Why Do We Need Containerization?

- Portability Across Platforms
- Efficiency and Resource Utilization
- Faster Start-up and Scalability
- Simplified Application Deployment
- Isolation and Security
- Supporting Microservices Architecture
- Integration with DevOps and CI/CD
- Hybrid and Multi-cloud Environments
- Container Orchestration for Large-Scale Deployments



Core Concepts in Containerization



Containers: Packages that include an application and its dependencies.



Images: Templates to create containers.



Registries: Repositories to store and share Docker images.



Runtime: Docker, containerd, runc.



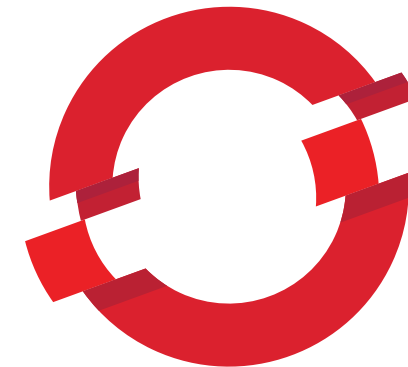
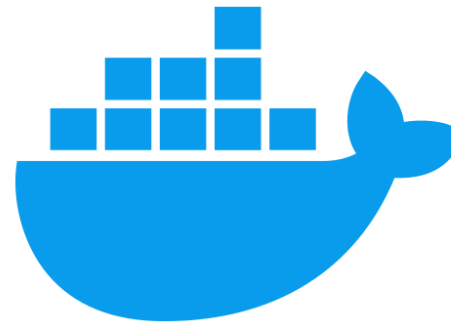
Major Container Platforms

Docker: Most widely used containerization platform.

Kubernetes: Orchestrates multiple containers across machines.

Podman: Docker-compatible alternative.

OpenShift: Kubernetes-based container platform.



OPENSIFT

Docker vs Other Container Platforms

Podman vs Docker

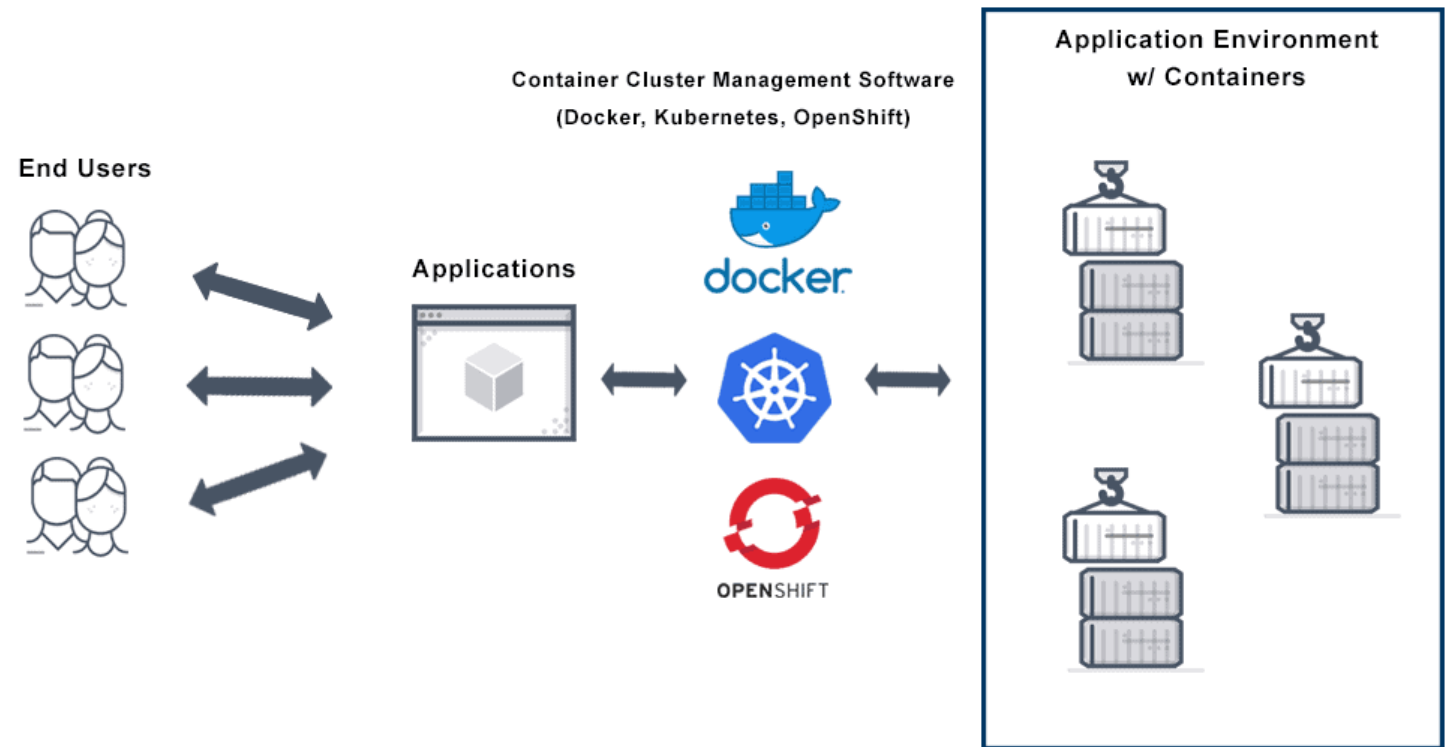
Aspect	Docker	Podman
Architecture	Client-server with a daemon	Daemonless, runs containers directly
Rootless Mode	Requires root permissions, potential security risk	Runs as a regular user, more secure
Security	Root-access daemon, single point of failure if compromised	No central daemon, reduced system-wide impact
Ease of Use	More community support, easier for beginners	Steeper learning curve, different management approach
Performance	Reliable, some overhead from daemon	Better performance, no daemon overhead

Docker vs Other Container Platforms

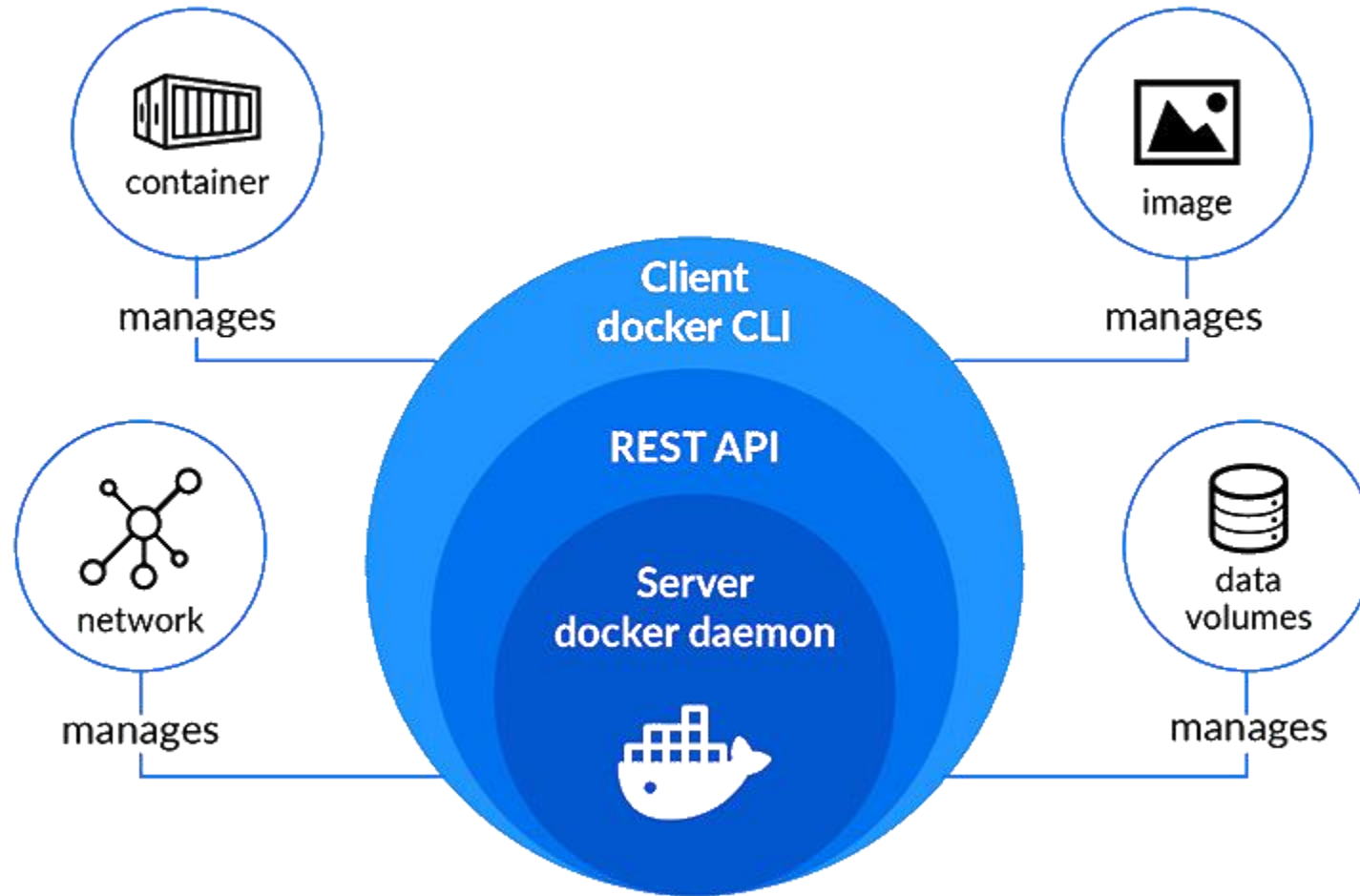
Feature	Containerd	Docker
Scope and focus	Minimalistic runtime for container orchestration	Comprehensive platform for containerisation
Abstraction level	Lower level of abstraction	Higher level of abstraction
Modularity	Follows a modular architecture	Monolithic platform
Dependency management	Minimal dependencies, lightweight	Includes additional components and dependencies
Community and ecosystem	Supported by the CNCF ecosystem	Large community supported by Docker, inc.

When to Use Docker, Kubernetes, or Other Tools?

- **Docker:** Best for single-machine container management.
- **Kubernetes:** Best for orchestrating large clusters.
- **Podman:** Good for running containers without a daemon.



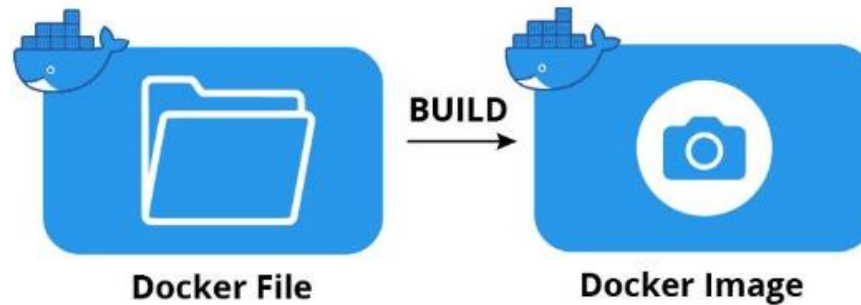
Docker Architecture



What is Docker Images and Containers?

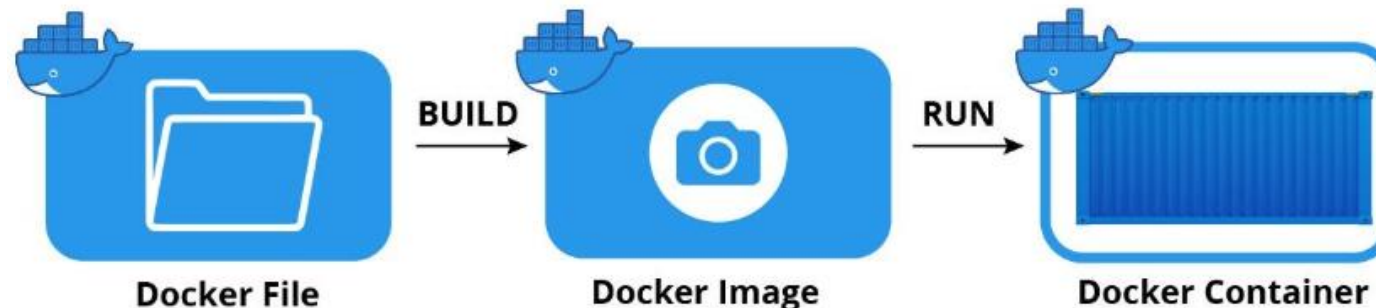
Image

An **image** is a **static, read-only file** that contains everything needed to run an application: the code, libraries, dependencies, environment variables, and configuration files.

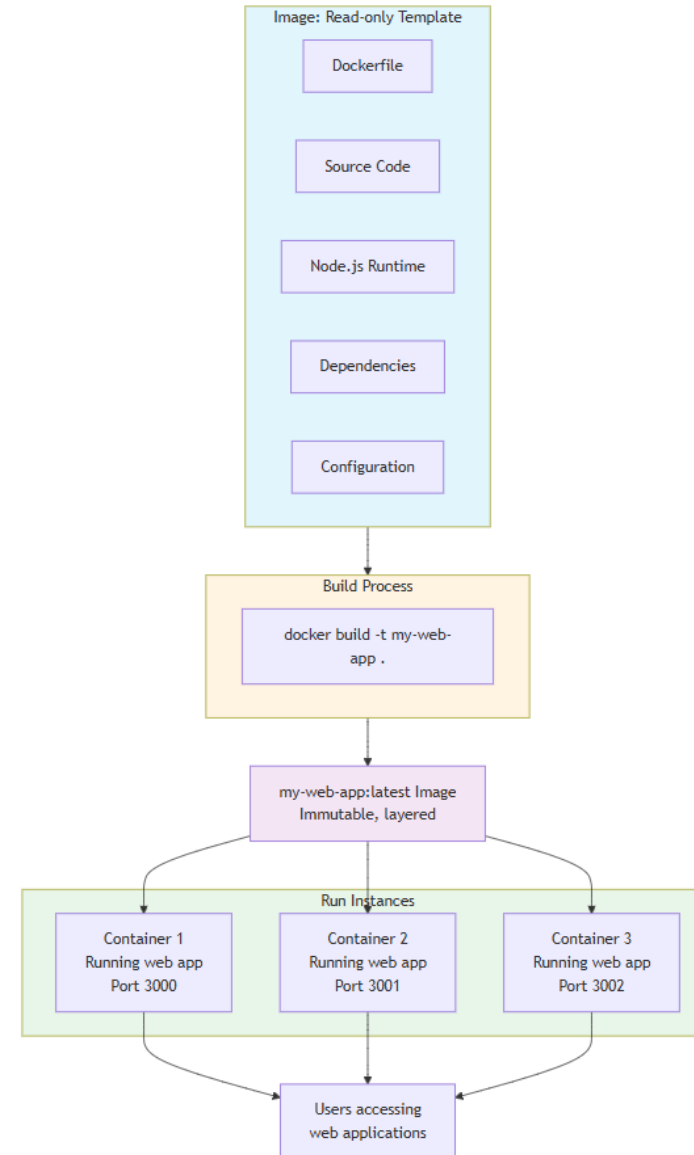


Container

A **container** is a **running instance** of an image.



What is Docker Images and Containers?

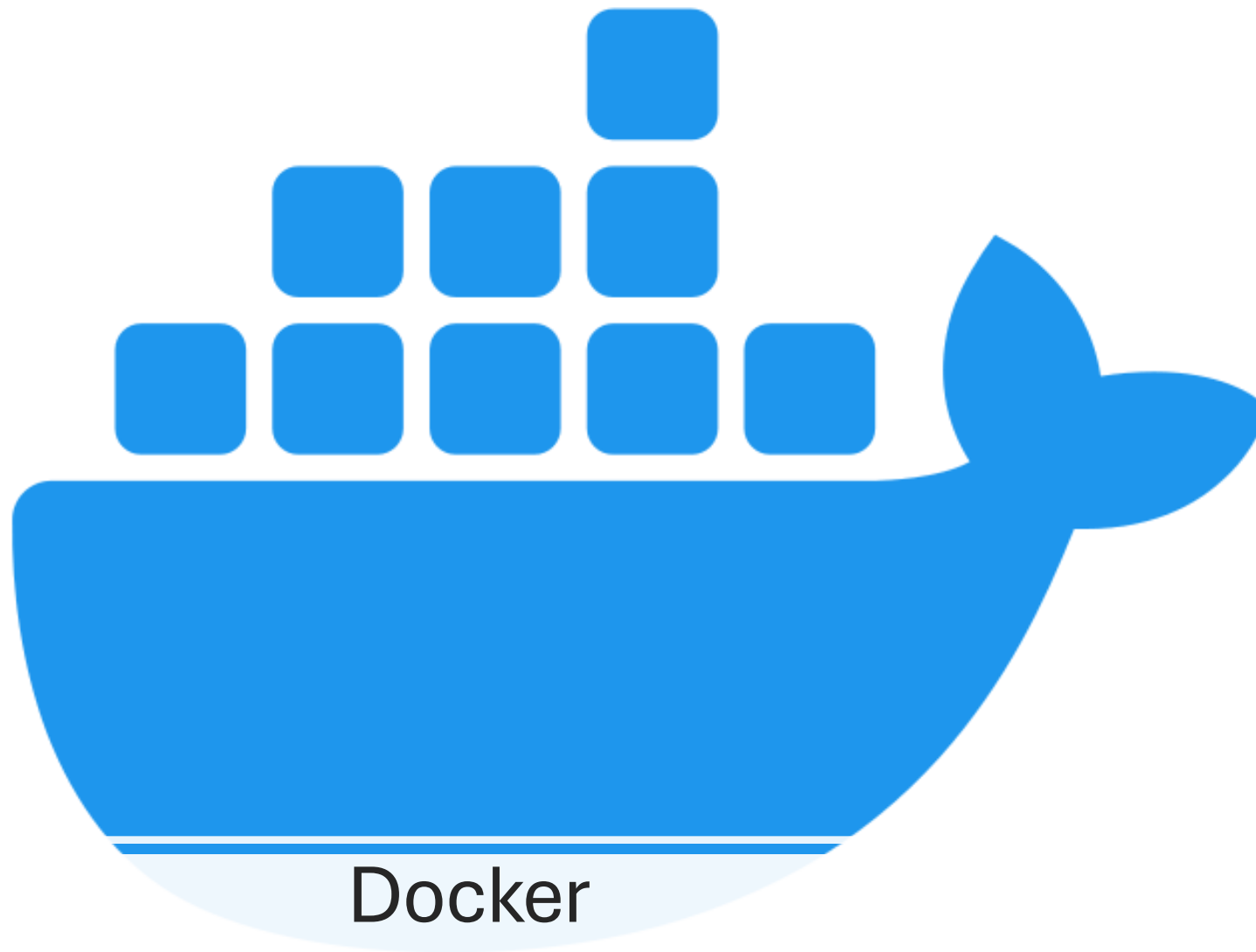




The image is the recipe; the container is the cake.

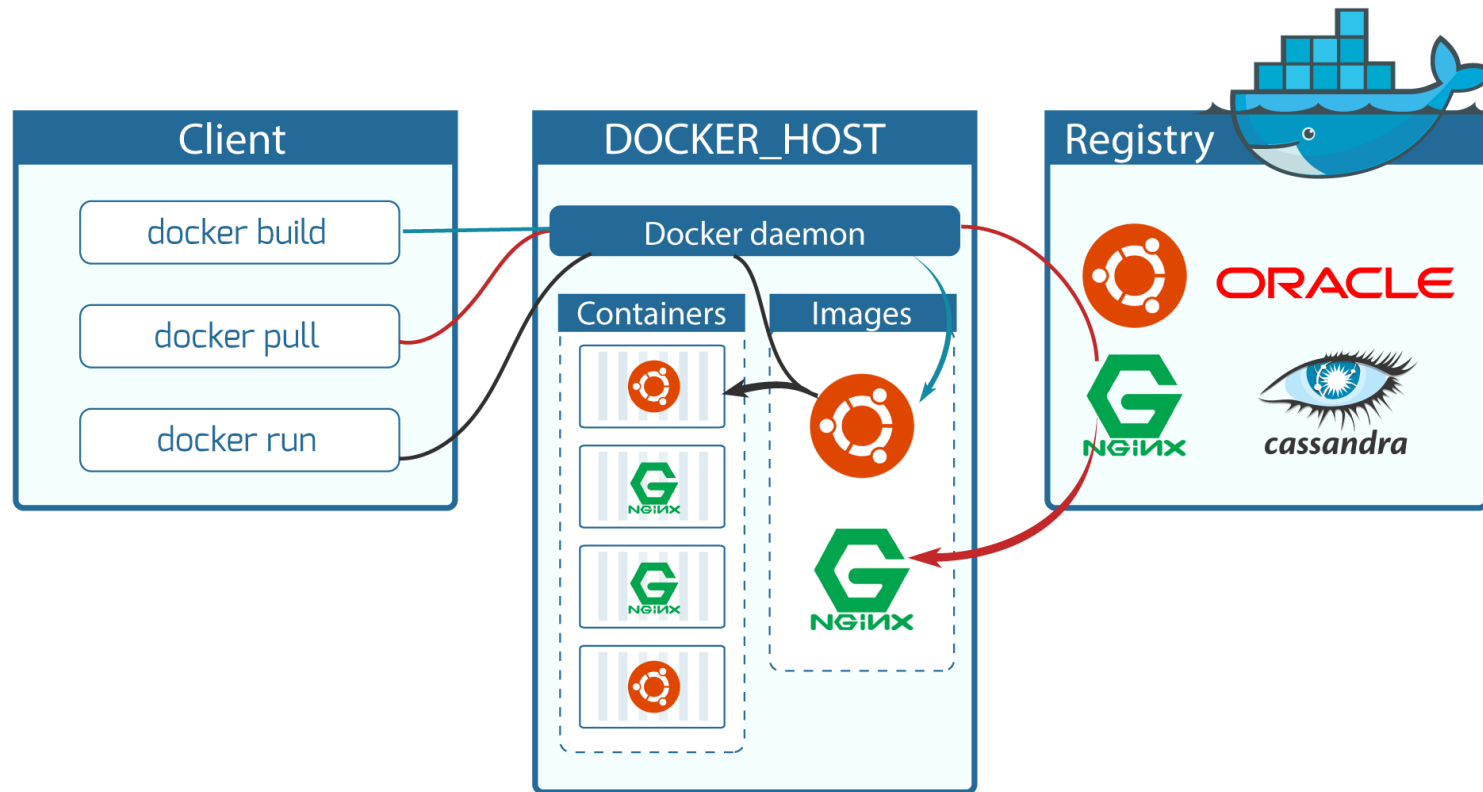
You can make as many cakes as you like with a given recipe





How Docker Works

DOCKER COMPONENTS



Dockerfile

```
1 FROM ubuntu:latest
2 RUN apt-get update -y
3 RUN apt-get install -y python-pip python-dev build-essential
4 COPY . /app
5 WORKDIR /app
6 RUN pip install -r requirements.txt
7 ENTRYPOINT ["python"]
8 CMD ["app.py"]
```


Docker Commands Overview

- `docker run`: Run a container.
- `docker ps`: List running containers.
- `docker stop`: Stop a running container.
- `docker build`: Build an image from a Dockerfile.

command	description
<code>docker create image [command]</code> <code>docker run image [command]</code>	create the container = <code>create</code> + <code>start</code>
<code>docker start container...</code> <code>docker stop container...</code> <code>docker kill container...</code> <code>docker restart container...</code>	start the container graceful ² stop kill (SIGKILL) the container = <code>stop</code> + <code>start</code>
<code>docker pause container...</code> <code>docker unpause container...</code>	suspend the container resume the container
<code>docker rm [-f³] container...</code>	destroy the container

Getting Started

- First Container

Run the following command:

```
$ docker run -ti ubuntu /bin/bash1
```

```
1  $ docker run -ti ubuntu /bin/bash
2      Unable to find image 'ubuntu:latest' locally
3      latest: Pulling from library/ubuntu
4      35c102085707: Pull complete
5      251f5509d51d: Pull complete
6      8e829fe70a46: Pull complete
7      6001e1789921: Pull complete
8      Digest:
9      sha256:d1d454df0f579c6be4d8161d227462d69e163a8ff9d20a847533989cf0c94d90
      Status: Downloaded newer image for ubuntu:latest
```

Getting Started

Many things happened when executing the above command.

1. Docker checks for the image in the local repository. If not found, it will look for it in the default Docker registry, DockerHub, and it will Pull (Download) the image and store it locally.
2. The Image is then started as a container with its file system, network, IPs, and Bridge interface.
3. Finally, it will execute the instructions provided. Since we ran the command with the options `-ti`, it will run in Terminal Interactive mode the command: `/bin/bash`

Now What?!!



Developer:

*It works on my
computer*



Product Manager:

*Yes, but we are not going
to give your computer to
the customer*



Pets Vs. Cattle

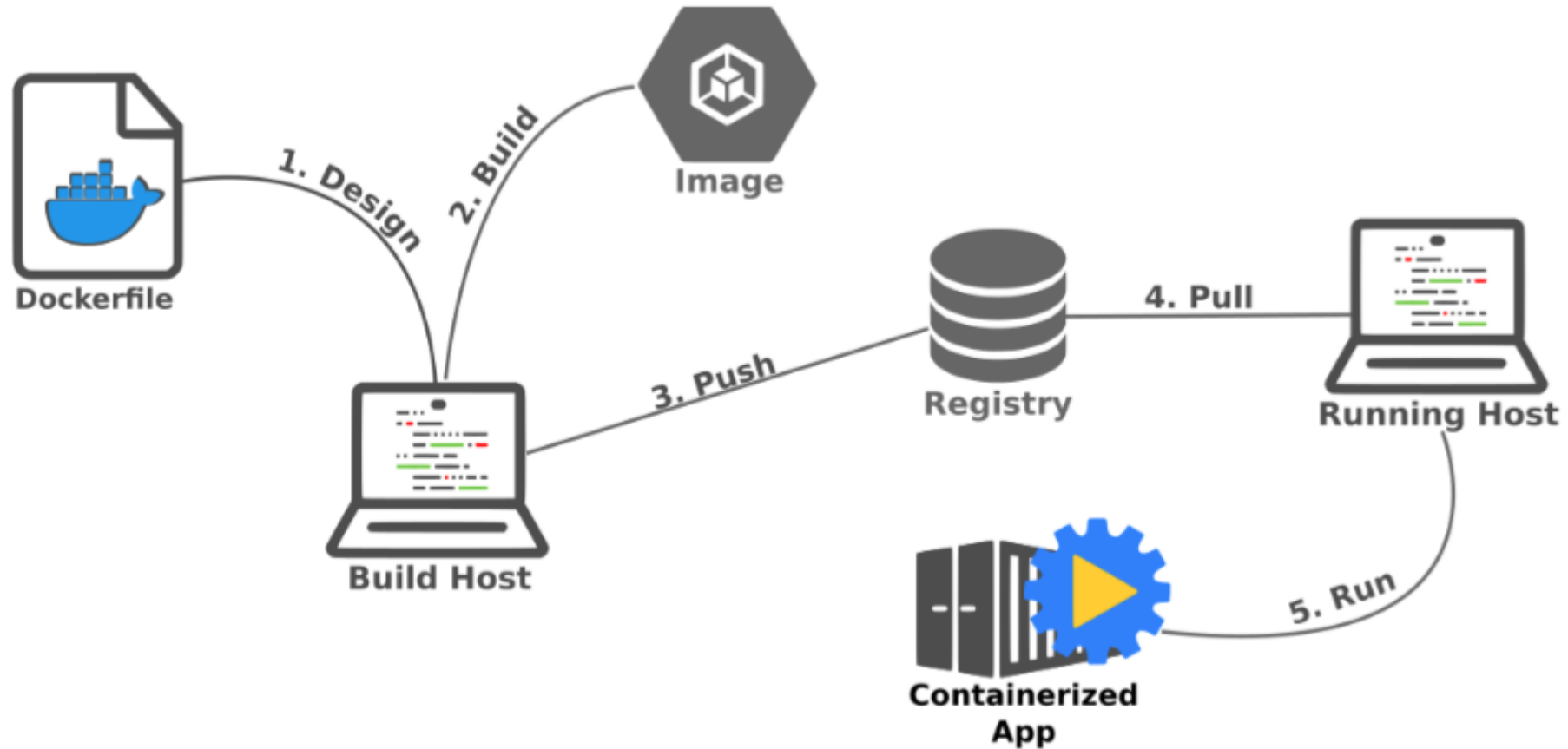
Containers are cattle's not pets



VS



Container creation workflow



Docker Client (CLI)

- Listing

```
1 $ docker ps -a
2 CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS
3 4b5263ee03a5   ubuntu    "/bin/bash"            2 hours ago   Exited (127)
19 seconds ago   quizzical_engelbart
4 ce35c4e6c746   fedora    "/bin/bash"            14 months ago Exited (0) 14
months ago       optimistic_kalam
5 fbbe7d946f4e   ubuntu    "sh"                   15 months ago Exited (0) 15
months ago       eager_pike
6 153d402cb0c2   fedora    "sh"                   15 months ago Exited (130)
15 months ago    wizardly_allen
7 b1da1e6e3ea8   hello-world "/hello"              16 months ago Exited (0) 16
months ago       eager_gauss
8 13149fb95bc8   hello-world "/hello"              16 months ago Created
```


Docker Client (CLI)

- Listing

`docker ps -a` list of all the containers that have been executed

`docker ps` list that are currently running

`docker ps -l` To show the last container executed

`docker ps -ls` The size of the containers with the `-s` option

Docker Client (CLI)

- Inspect

The docker inspect command will interrogate our container and return its configuration information, name, commands, network configuration, and more valuable data.

```
1  docker inspect 4b5263ee03a5
2  [
3      {
4          "Id":
5      "4b5263ee03a5e7d83a9916eb194c59759de97852d9ec231cb53b9e0988ae2535",
6          "Created": "2021-06-28T20:35:50.587156443Z",
7          "Path": "/bin/bash",
8          "Args": [],
9          "State": {
10             "Status": "exited",
11             "Running": false,
12             ...
13         }
14     }
15 ]
16 -Output_truncated
```

Docker Client (CLI)

- Inspect

you can use it with the `--format` flag to limit the amount of data returned as in the following examples:

```
1 docker inspect --format='{{.State.Running}}' daemon_alice
2 docker inspect --format='{{.NetworkSettings.IPAddress}}' daemon_alice
3 docker inspect --format='{{.Name}} {{.State.Running}}' daemon_alice
4 $ docker inspect --format='{{.State.Running}}' 4b5263ee03a5
```

We can list multiple containers by listing them.

```
1 docker inspect --format='{{.Name}} {{.State.Running}}' daemon_alice
   daemon_dan daemon_juan
```

Docker Client (CLI)

- Process Management

We can monitor the processes running inside of a container with top

```
1 $ docker top a7cc3837caf4
2 UID          PID         PPID        C         STIME        TTY         TIME         CMD
3 root         22397      22376       0         18:35        ?          00:00:00    /bin/sleep 5000
```

You can also use stats to see CPU MEM IO and more.

```
1 $ docker stats
2 CONTAINER ID   NAME          CPU %       MEM USAGE / LIMIT   MEM %
3 a7cc3837caf4   hardcore_moore 0.00%      388KiB / 15.34GiB   0.00%
4.43kB / 0B    73.7kB / 0B      1
```

Docker Client (CLI)

- Monitoring

We can see what is going on inside a container with logs

```
1 | $ docker logs daemon_juan
```

We can also simulate a tail -f with the -f flag. Stop with Ctrl+C

```
1 | $ docker logs -f daemon_juan
```

or with --tail

```
1 | $ docker logs --tail 10 daemon_juan
```

We can also add timestamps to our output -t

```
1 | $ docker logs -ft daemon_juan
```

Docker Client (CLI)

- Naming

Docker will automatically generate a name for each container. But if we want to specify a name, we use:

```
1 | $ docker run --name contained_bob -ti ubuntu /bin/bash
```

- Only characters and numbers are allowed
- Names must be unique



THE DOCKER CONTAINER



MY HOOT MACHINE

Docker Client (CLI)

- Starting

There is not much to say about it. If the container exist, it will start!

```
1 | $ docker start mycontainer
```

- Creating

Nevertheless, if the container does not exist, it needs to be created first.
create Creates the container and all its dependencies but does not run it.

```
1 | $ docker create --name mycontainer ubuntu  
2 | 50d16cb60fdc2ba4a1a0c08919b6a17b7b1488cd78390922c9ad14cb8bf998f1
```


Docker Client (CLI)

- Exec

We can run new processes on existing containers with exec. There are two types of commands we can run:

- Background and Interactive
- Background & Foreground processes!

```
1 # Non-Interactive command [Simple]
2 $ docker exec daemon_dan ls -l /etc/new_config_file
3 # Background command [Daemonized]
4 $ docker exec -d daemon_dan touch /etc/new_config_file
5 # Interactive command [Interactive]
6 $ docker exec -ti daemon_dan /bin/bash
```

Docker Client (CLI)

- Stopping

This sends a SIGTERM signal to the container running process.

```
1 | $ docker stop daemon_juan
```

You can also send a SIGKILL with

```
1 | $ docker kill daemon_juan
```

We can check N number of recently stopped containers with

```
1 | $ docker ps -n 10
```

Docker Client (CLI)

- Restarting

You can use the `--restart` flag to restart a container automatically. It does it by checking the exit code and making a decision. The default behavior is not to restart.

```
1 | $ docker run --restart=always --name daemon_alice -d ubuntu /bin/sh -c  
    "while true; do echo hello alice; sleep 1; done"
```

Options:

- on-failure This also accepts an optional count max: on-failure:5 attempting to restart to a max of 5 times
- always

Docker Client (CLI)

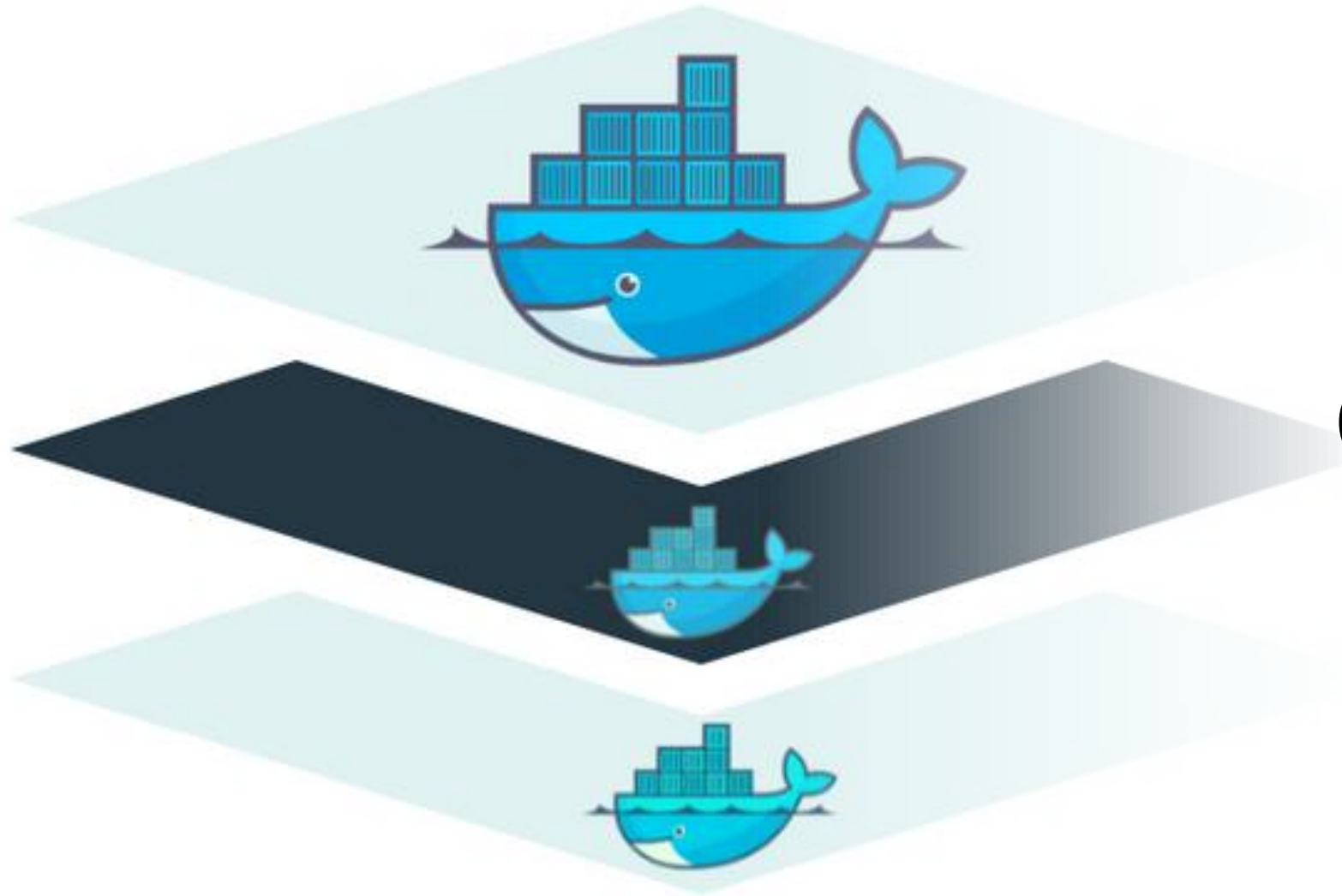
- Deleting

You can use the `rm` command to remove a container definitively. If the container is running to force deleting it, you can add the `-f` flag!

```
1 $ docker rm daemon_alice
2 # Error response from daemon: You cannot remove a running container
  70a3dcabd2b022deaaa804c68dc7b2e9185fe9388f221cb62e28848ed06ea165. Stop
  the container before attempting removal or force remove
```



Finally Over!

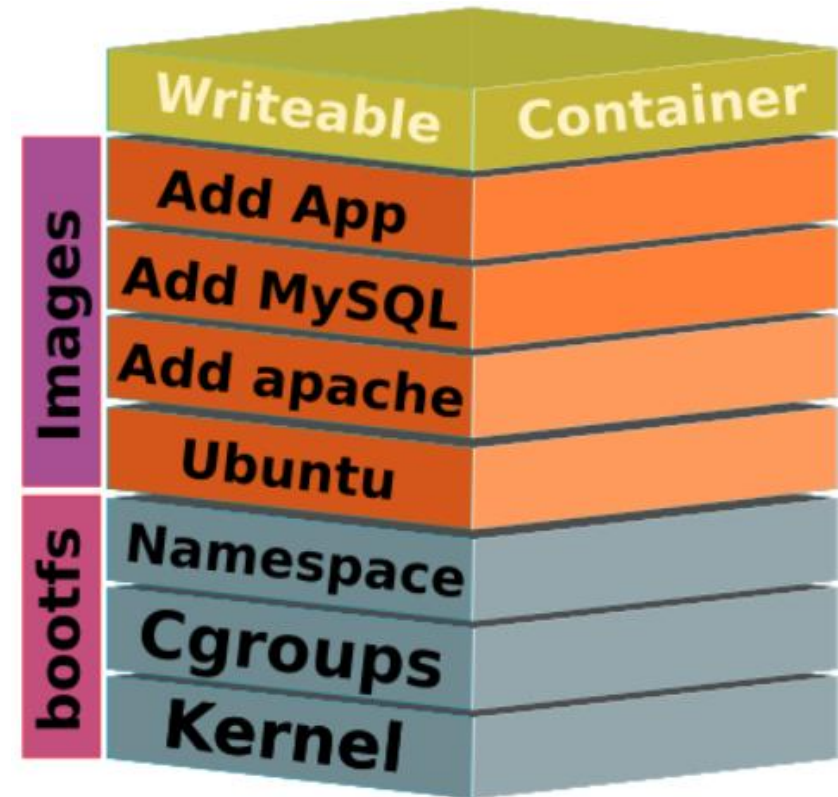


Container Images

Docker Image

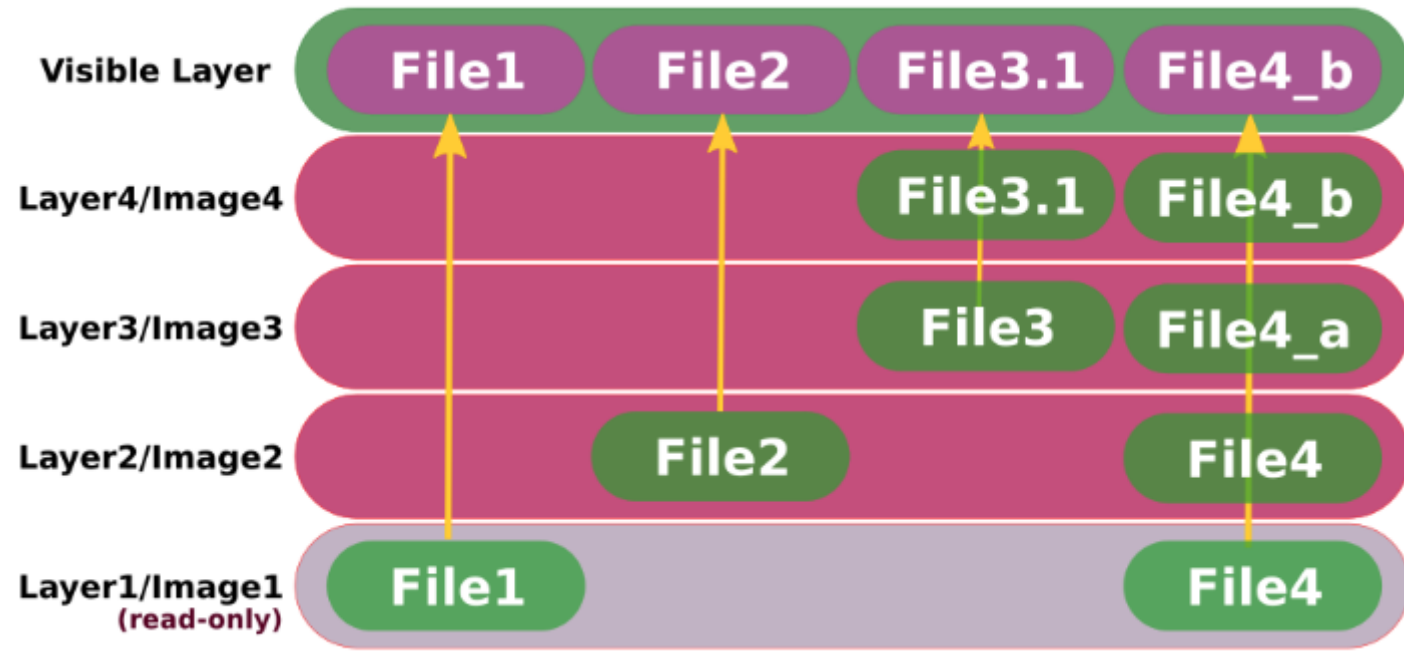
A Docker Image is made of file-systems layered over each other.

- The base is a boot file-system boot-fs similar to the one Linux uses.
- Followed by a root file system root-fs/kernel, this holds the operative system layer, different from Linux.
- This one remains in read-only mode, and Docker uses the UFS union filesystem to add more read-only file-systems on top of it.



Docker Image

Docker calls each one of these layers images. The image below is called parent image . You can move around them until you get to the bottom, called base image.

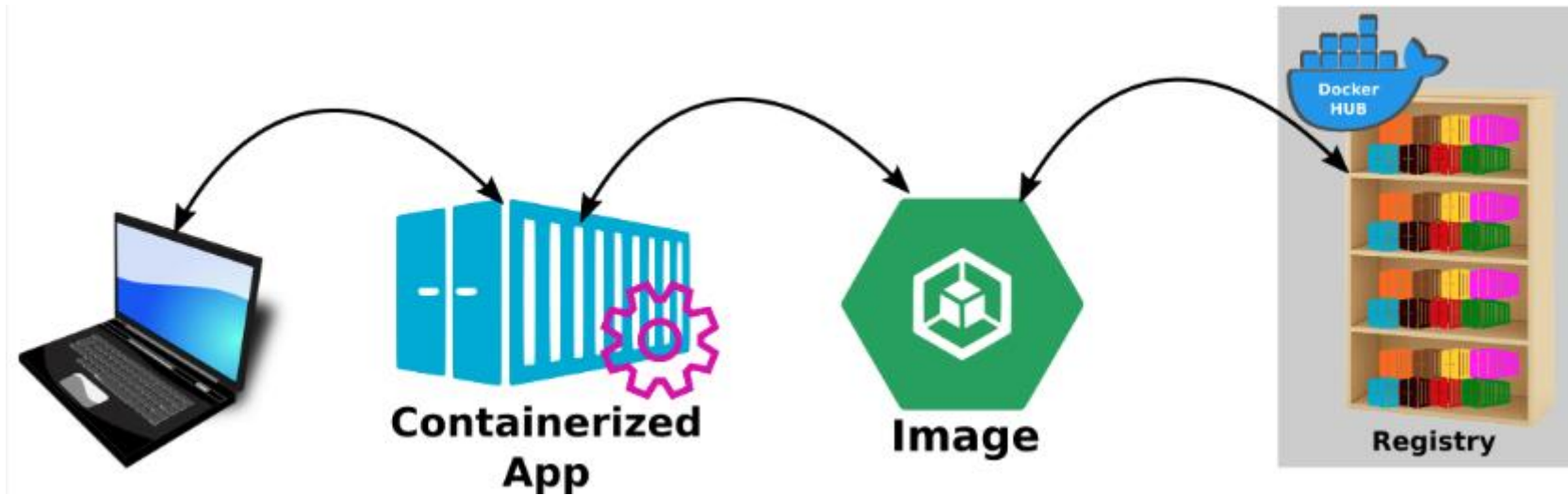


Managing Docker Images

Command	Description
<code>docker build</code>	Build an image from a Dockerfile
<code>docker commit</code>	Create a new image from a container's changes
<code>docker history</code>	Show the history of an image
<code>docker images</code>	List images
<code>docker inspect</code>	Display detailed information on one or more images
<code>docker pull</code>	Pull an image or a repository from a registry
<code>docker push</code>	Push an image or a repository to a registry
<code>docker rmi</code>	Remove one or more images
<code>docker search</code>	Search the Docker Hub for images
<code>docker tag</code>	Create a tag <code>TARGET_IMAGE</code> that refers to <code>SOURCE_IMAGE</code>
<code>docker image prune</code>	Remove unused images

Registry

The registry is stored to control and distribute images



There are two types of repositories:

1. User Repositories: Images contributed by Docker users. (Use them with caution!!!).
2. Top-Level Repositories: Controlled by Docker.

Login

Register for a Docker ID Before you can start using the services of Docker, you have to register to get a Docker ID.

1. Go to the Docker Hub signup page.
2. Enter a username that is also your Docker ID.

Your Docker ID must be between 4 and 30 characters long and only contain numbers and lowercase letters.

1. Enter a unique, valid email address.
2. Enter a password. Note that the password must be at least 9 characters.
3. Complete the Captcha verification and then click Sign up. Docker sends a verification email to the address you provided.
4. Verify your email address to complete the registration process.

Note: You cannot log in with your Docker ID until verifying your email address.

```
1 $ docker login
2 Username: jmedinar
3 Password:
4
5 Login Succeeded
```

```
1 $ docker logout
```

PULL

Will download images if they are not already present in the host.
This saves time when launching containers.

```
1 $ Docker pull ubuntu
2 Using default tag: latest
3 latest: Pulling from library/ubuntu
4 c549ccf8d472: Pull complete
5 Digest:
   sha256:aba80b77e27148d99c034a987e7da3a287ed455390352663418c0f2ed40417fe
6 Status: Downloaded newer image for ubuntu:latest
7 docker.io/library/ubuntu:latest
```

PUSH

Once you complete creating and testing your image, you are ready to make it public if you want to by pushing it to DockerHub, making it available for other users.

```
1 $ docker push jmedinar/my-new-apache2
2 Using default tag: latest
3 The push refers to repository [docker.io/jmedinar/my-new-apache2]
4 fc02d4dcd423: Pushed
5 e80c789bc6ac: Mounted from jmedinar/my_custom_app
6 6c3332381368: Mounted from jmedinar/my_custom_app
7 ef1a1ec5bba9: Mounted from jmedinar/my_custom_app
8 a1aa3da2a80a: Mounted from jmedinar/my_custom_app
9 latest: digest:
  sha256:0998da23dae955b5edd96a9f8c0b7f2a9e441a8cd3716585ccf9803362d249fd
  size: 1364
```

Container Files

Dockerfile

Docker can build images automatically by reading the instructions from a Dockerfile. A Dockerfile is a text document that contains all the commands a user could call on the command line to assemble an image. Using `docker build`, users can create an automated build that executes several command-line instructions in succession.

- Dockerfile is the default name but can have any other name.
- Must exist in the root context of the project we are creating.

Dockerfile

Command	Purpose
FROM	To specify the base image which we want to use.
WORKDIR	To define the working directory for any commands that follow in the Dockerfile.
RUN	To install a package or any application.
COPY	To copy over files or directories from a specific location
ADD	Same as COPY, but we can also use a URL instead of a local file / directory and we can extract a tar file from the source directly into the destination.
ENTRYPOINT	Command that will always be executed when the container starts. If not specified, the default is <code>/bin/sh -c</code>
CMD	To define a default command to run when your container starts.
EXPOSE	To define which port through which to access your container application.
LABEL	To add metadata to the image.

Dockerfile

- Sample

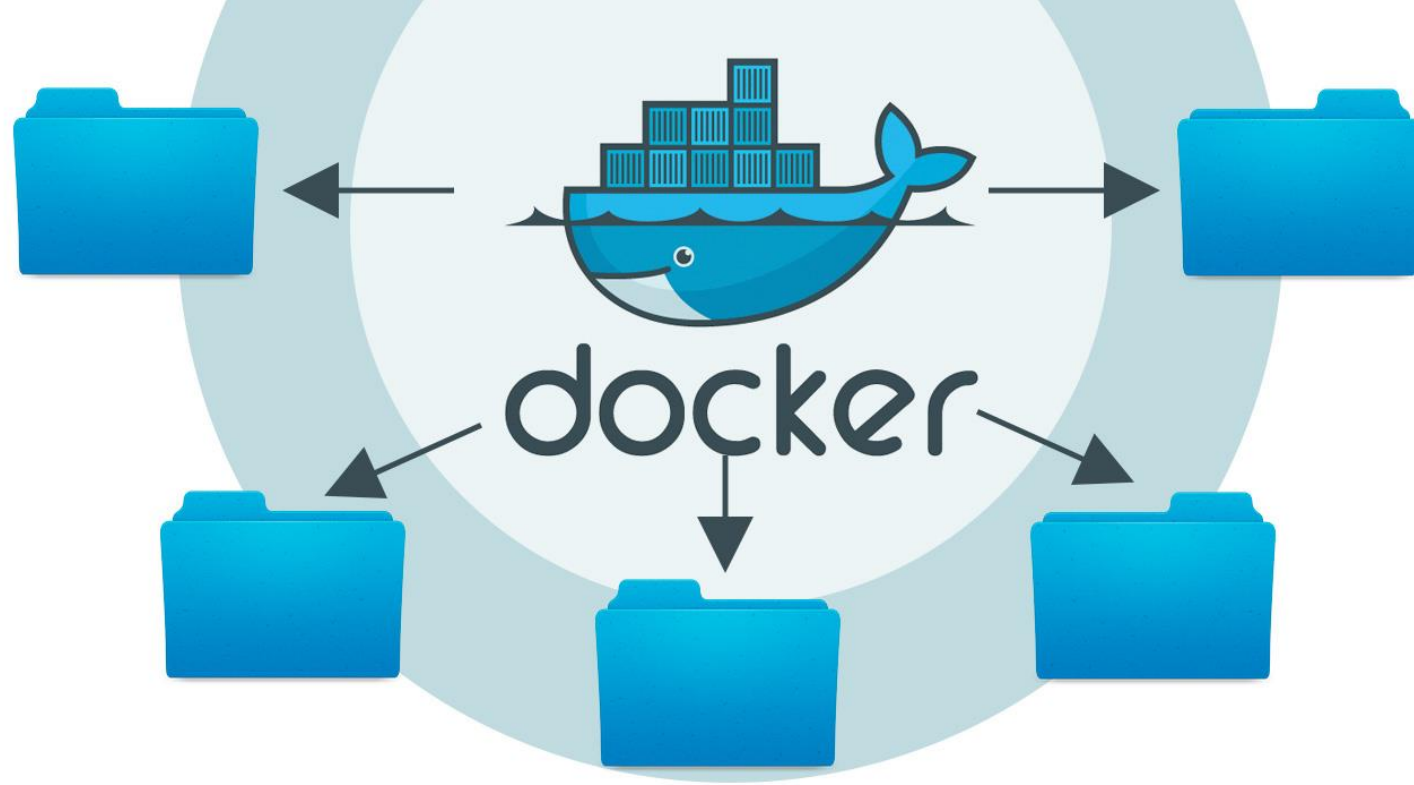
```
# Use a lightweight Python image
FROM python:3.8-slim

# Set working directory
WORKDIR /app

# Copy the controller code into the container
COPY controller2.py /app/controller2.py

# Install dependencies
RUN pip install kubernetes pyyaml

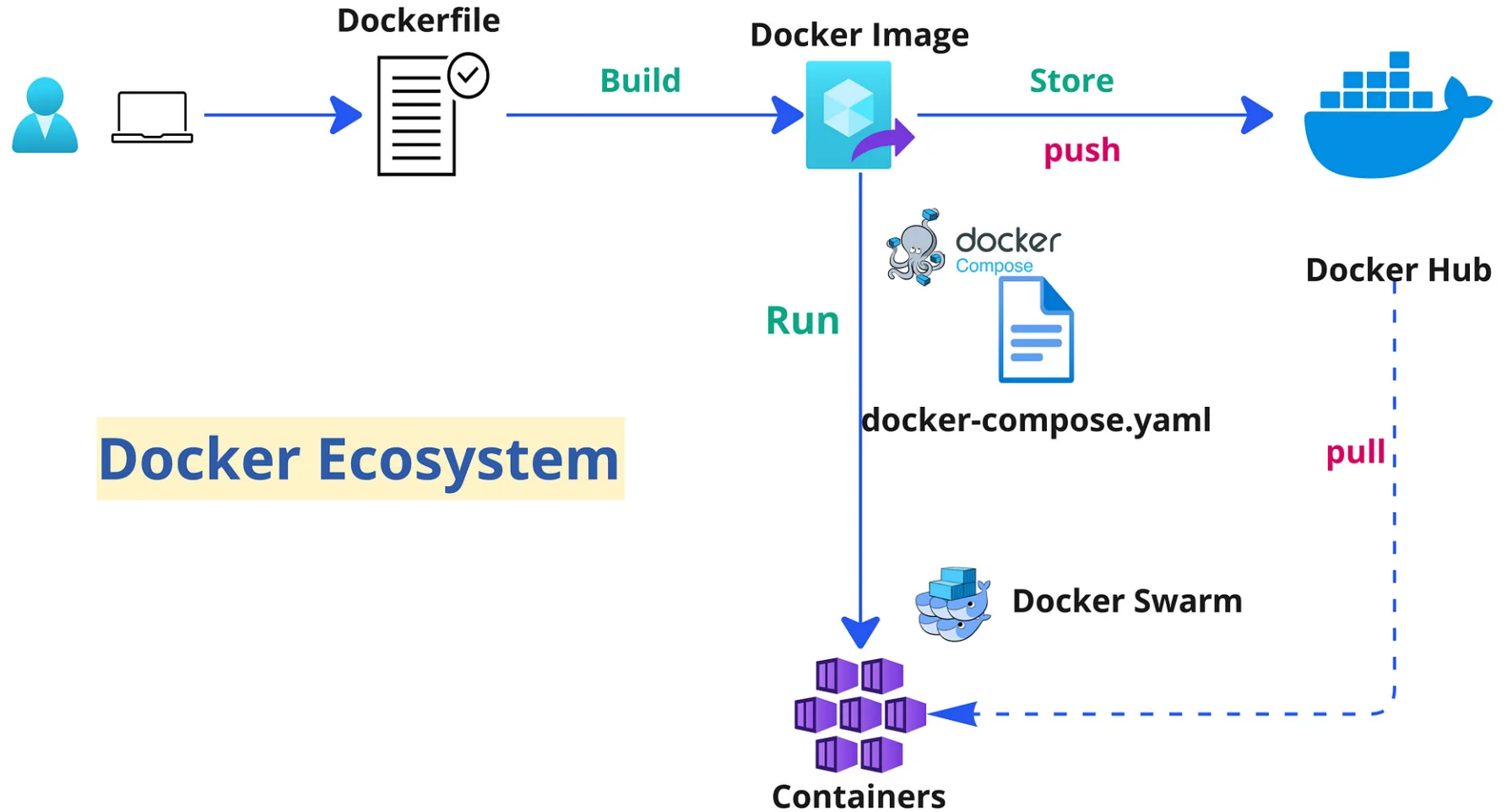
# Run the controller script
CMD ["python", "/app/controller2.py"]
```



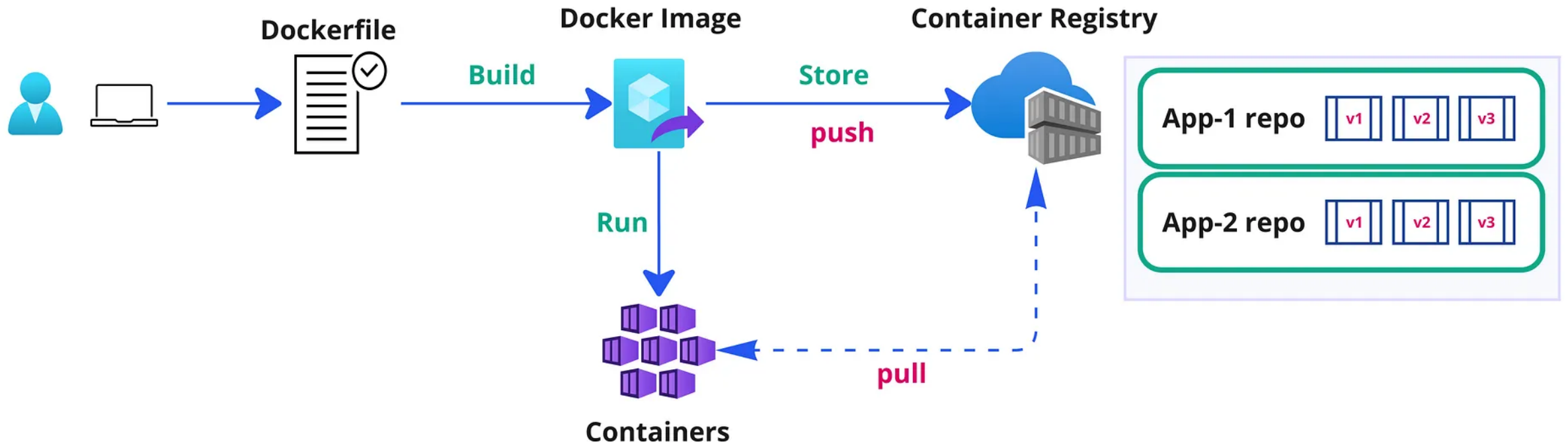
Docker Storage

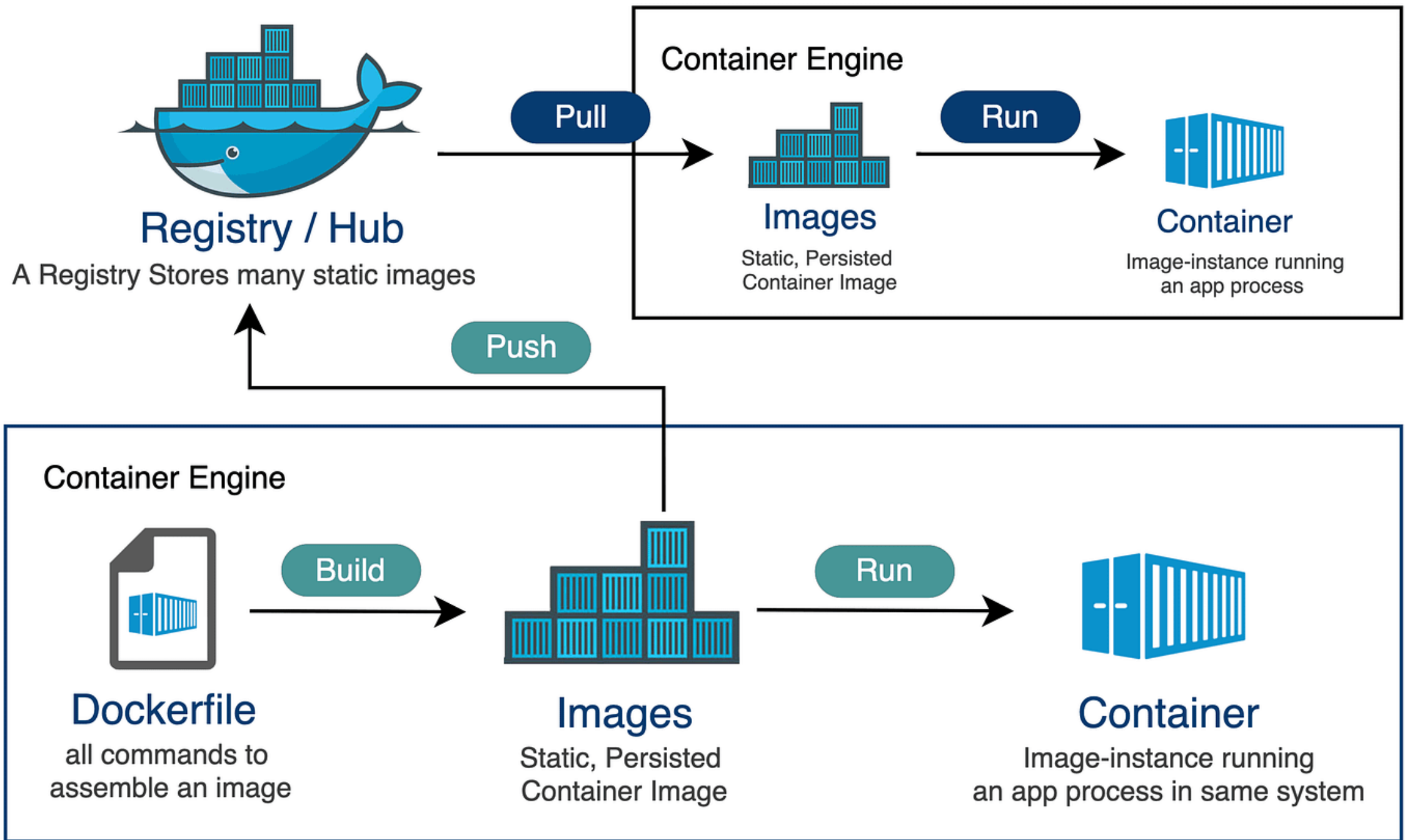
Containers Properties

- Ephemeral
- Idempotent
- Immutable



What we did so far





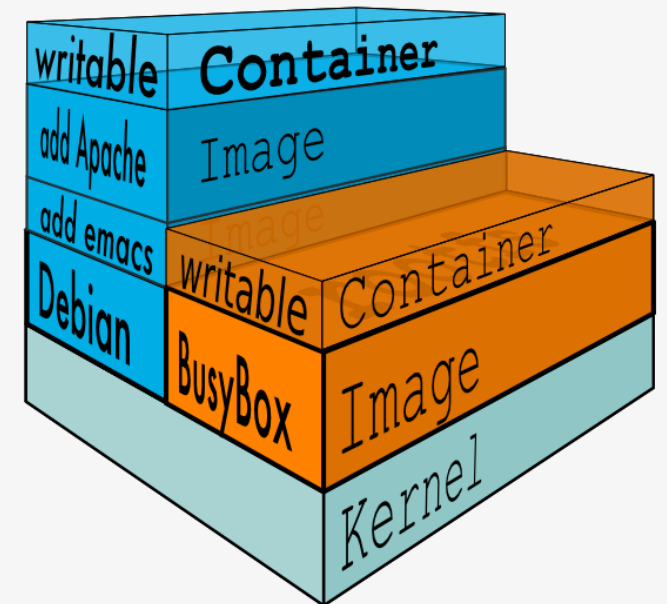
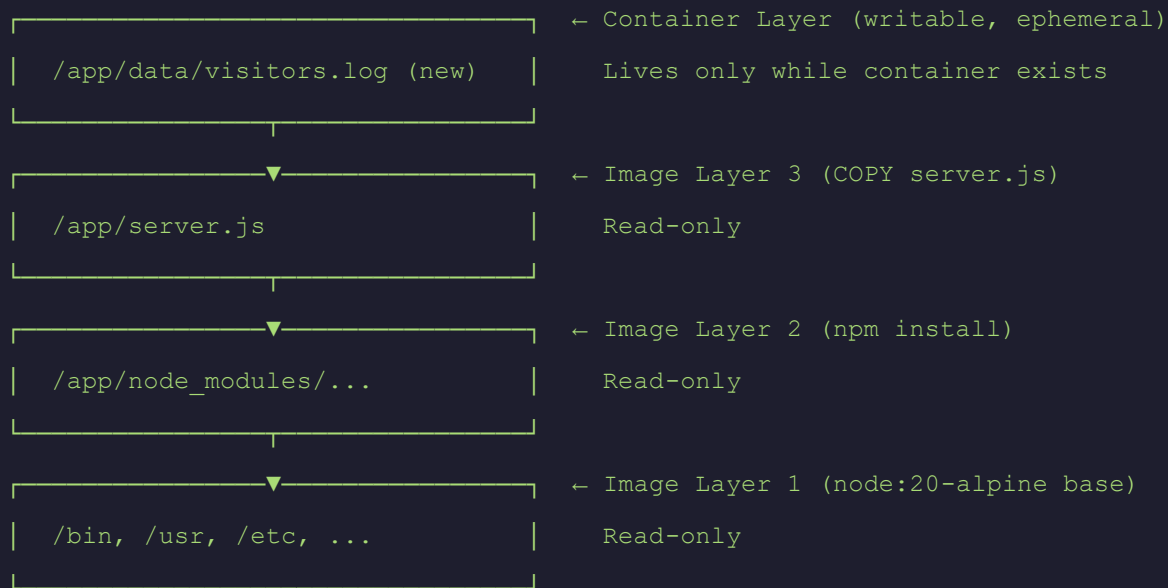
Docker Storage Options

Docker containers are ephemeral by design, when a container is removed, everything written inside it disappears. This is intentional: it keeps containers stateless, portable, and reproducible.

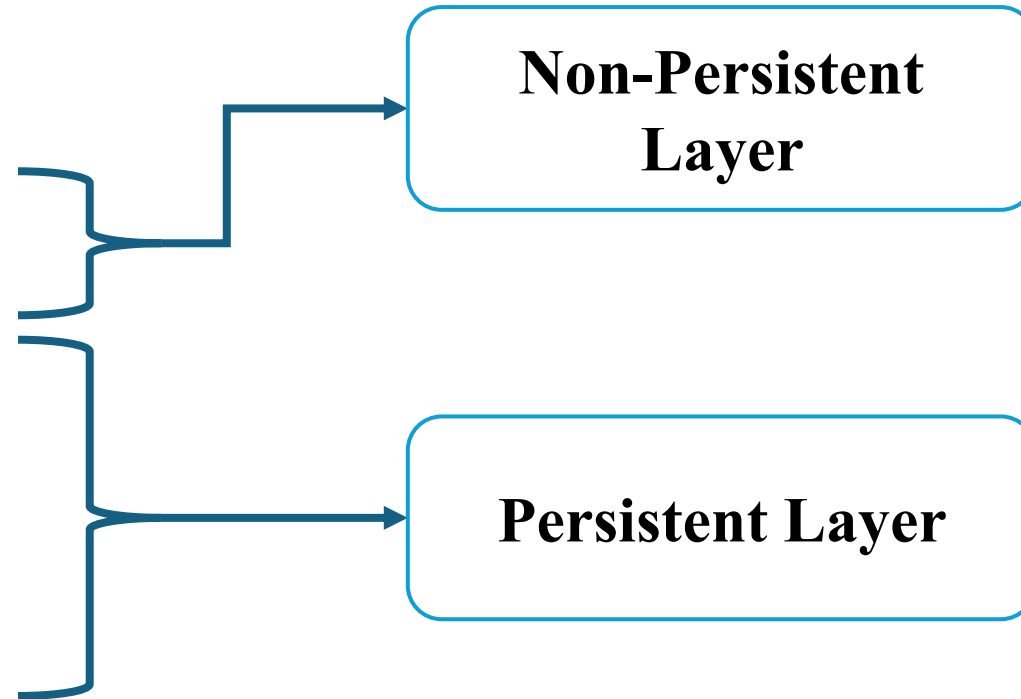
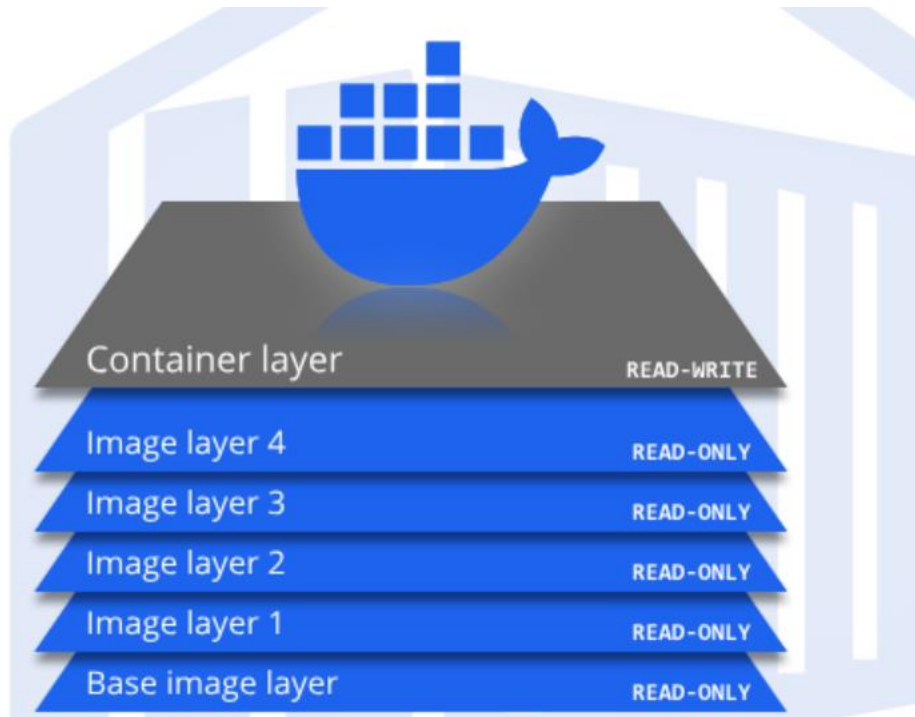
Storage Type	Managed By	Persists After rm?	Best For
Container Layer	Docker (ephemeral)	✗ No	Throwaway scratch data
Bind Mount	Host OS	✓ Yes	Dev workflows, config files
Named Volume	Docker Engine	✓ Yes	Production databases, logs

Docker – Union File System (Overlay FS)

Architecture Diagram (text representation)



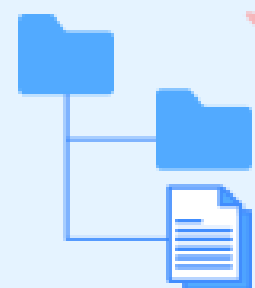
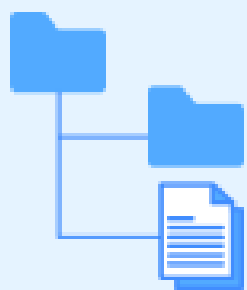
Docker Storage



Volume



Docker Container



volume

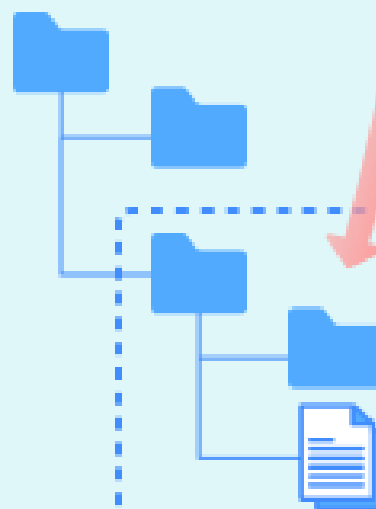


docker engine

Bind Mount

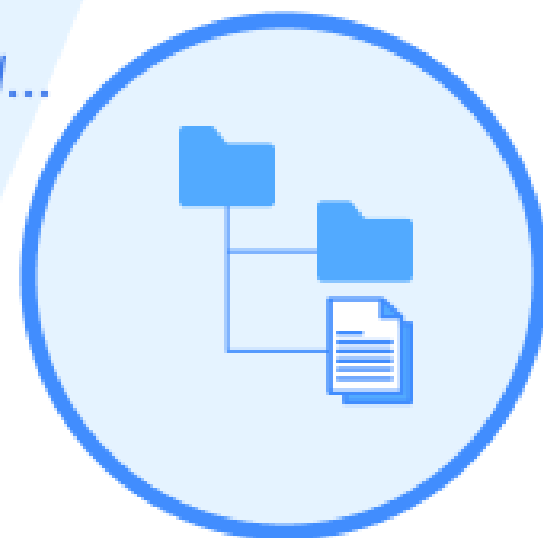
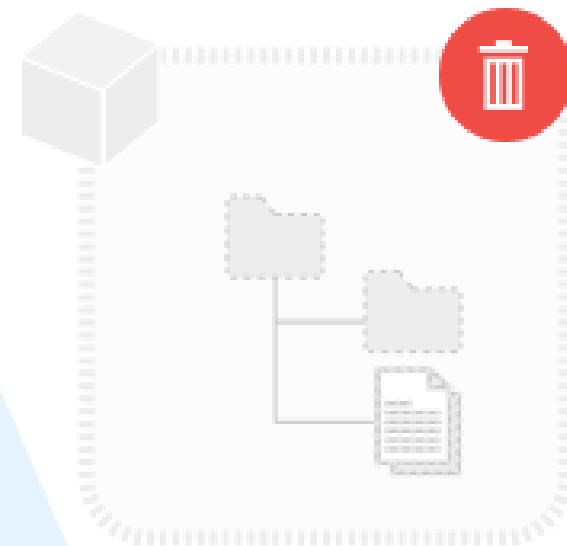


Bind Mount



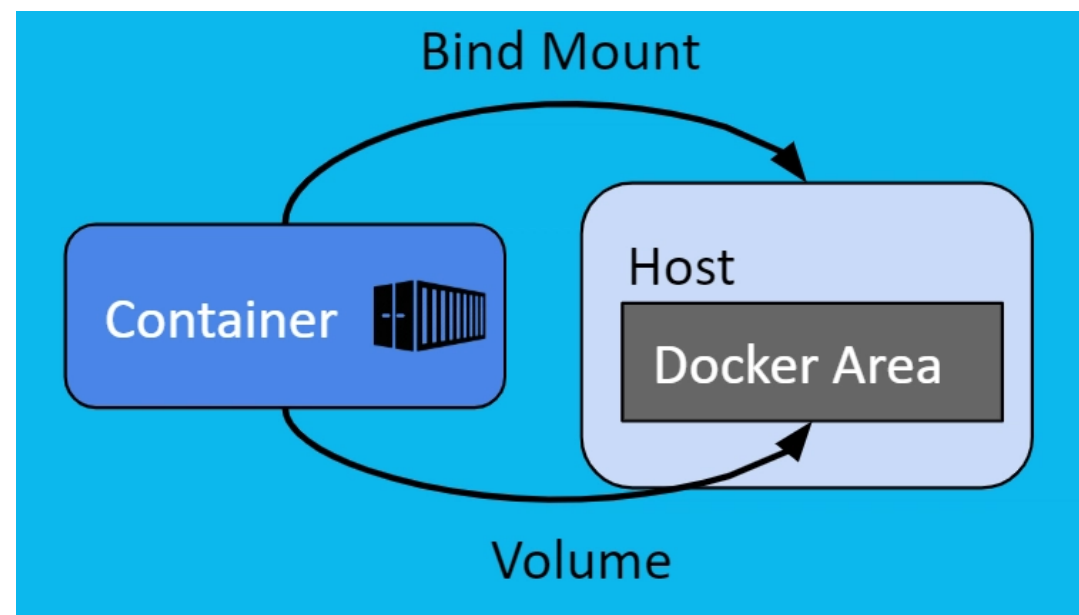
Host's file system

When
containers
are
removed...

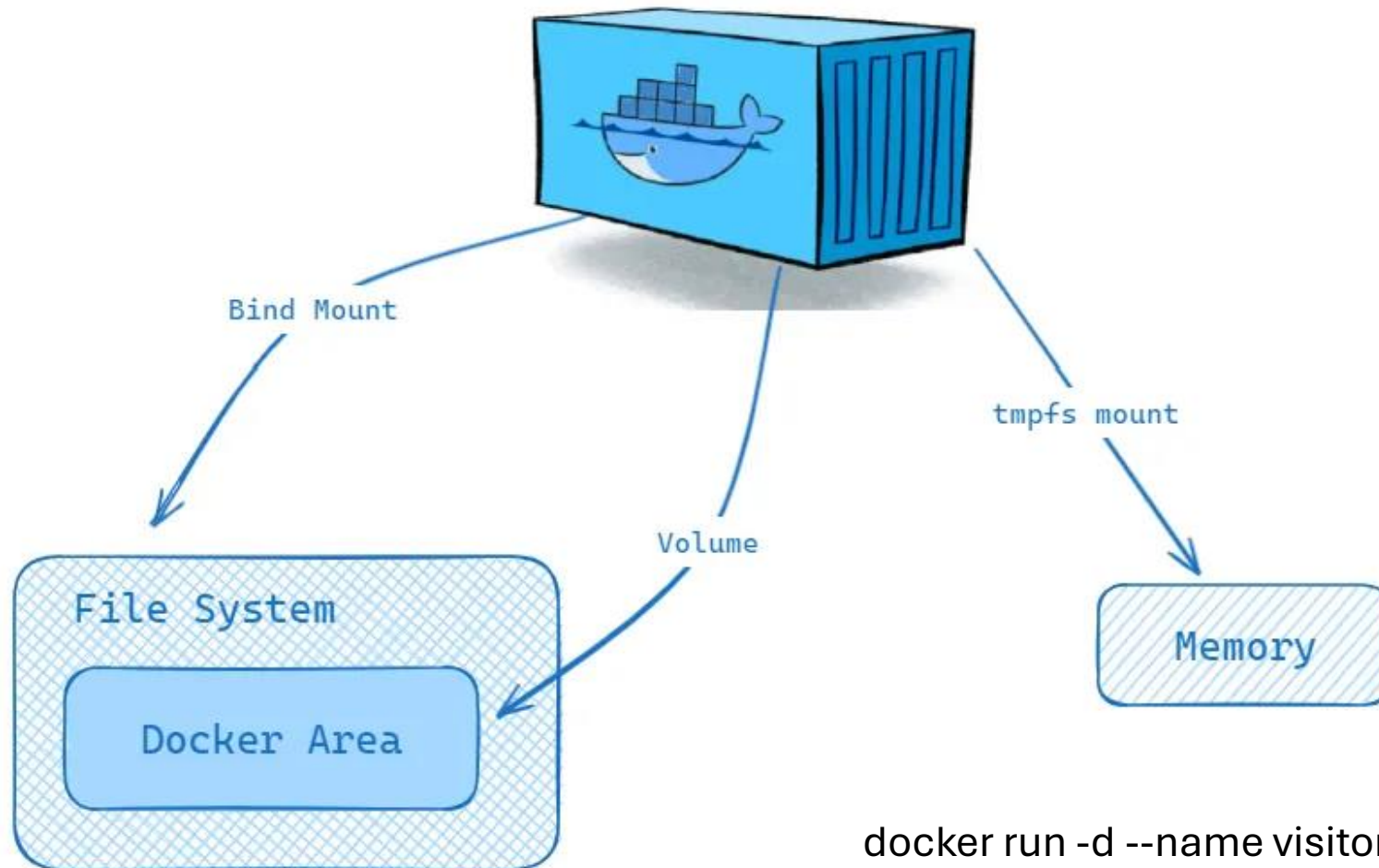


Docker Bind Mount

- `docker run -d --name visitorlogapp -p 3000:3000 --mount type=bind,src=D:\ClassDemo\logs,dst=/app/data visitorlog`

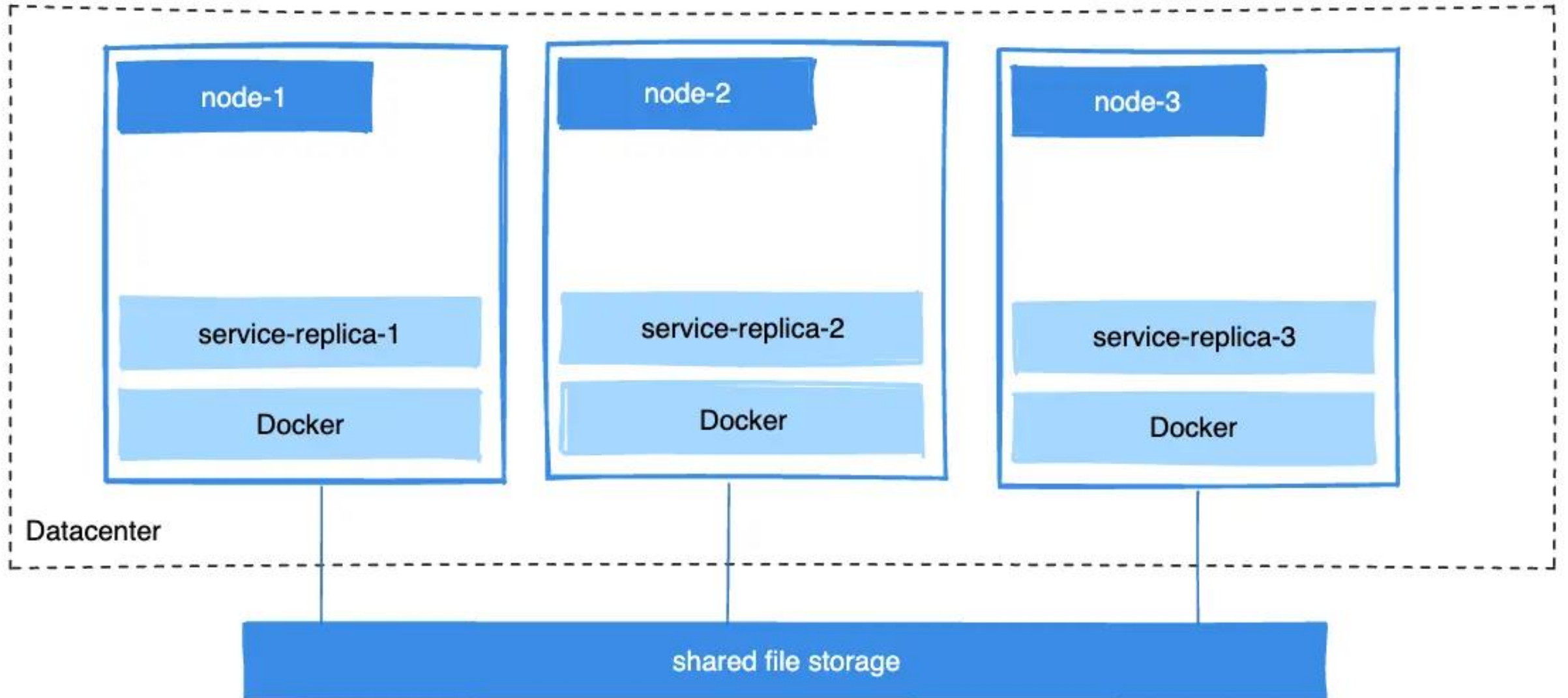


Docker Volume



```
docker run -d --name visitorwebapp -p 3000:3000 -v  
visitor_data:/app/data visitorlog
```

Sharing Data Between Containers



DEMO

Creating and running mysql container

- Pull MySQL image from Docker hub
 - `Docker pull mysql:latest`
- Run the mysql Docker container
 - `docker run --name mysql_db -e MYSQL_ROOT_PASSWORD=admin -p 3333:3306 -d mysql:latest`
- Go inside the container
 - `docker exec -it mysql_db sh`
 - `mysql -p`
 - `create database sakila;`
 - `exit mysql`
 - `exit container`
- Now copy the .sql dump file into container repo
 - `docker cp sakila.sql mysql_db:/tmp/sakila.sql`
- Now go again inside the docker container
 - `Mysql -p sakila < /tmp/sakila.sql`

Web Application + MySQL

- Create a Network
 - `docker network create networkmysql`
- Create a MySQL container from Pulled image
 - `docker run -d --name mysql-container -e MYSQL_ROOT_PASSWORD=admin -p 3307:3306 --network networkmysql mysql:latest`
- Create a Flask App container from newly created image
 - `docker run -d --name sakila-container --network networkmysql -p 5000:5000 sakila`